

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NHA TRANG
KHOA CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU CÀI ĐẶT VÀ CẢI TIẾN THUẬT TOÁN PHÂN CỤM
ĐỈNH MẬT ĐỘ DỰA TRÊN TỐI ƯU THAM SỐ**

Giảng viên hướng dẫn: TS. Nguyễn Thủy Đoan Trang
Sinh viên thực hiện: Trương Ngự Quyền
Mã số sinh viên: 64131995

Khánh Hòa – 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC NHA TRANG
KHOA CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU CÀI ĐẶT VÀ CẢI TIẾN THUẬT TOÁN PHÂN CỤM
ĐỈNH MẬT ĐỘ DỰA TRÊN TỐI ƯU THAM SỐ**

GVHD: TS. Nguyễn Thủy Đoan Trang

SVTH: Trương Ngự Quyền

MSSV: 64131995

Khánh Hòa - Tháng 6/2026

KẾT QUẢ ĐẠO VĂN CỦA BÁO CÁO

DoAn_TruongNguQuyên_NghienCuuCaiDatVaCaiTienThuatT...

BÁO CÁO ĐỌC SÁNG



NGUỒN CHÍNH

1	Submitted to Nha Trang University Bài của Học sinh	3%
2	www.ctu.edu.vn Nguồn Internet	2%
3	gust.edu.vn Nguồn Internet	1%
4	lib.uet.vnu.edu.vn Nguồn Internet	1%
5	Giang Lê Khánh, Hương Hồ Thị Lan, Linh Nguyễn Thuỳ. "Khung học máy lai ghép IF – LOF để phát hiện số liệu GNSS-RTK ngoại lai trong quan trắc chuyển dịch cầu dây văng", Transport and Communications Science Journal, 2026 Xuất bản	1%
6	Ton Duc Thang University Xuất bản	1%
7	Hanoi National University of Education Xuất bản	1%

TRƯỜNG ĐẠI HỌC NHA TRANG

Khoa: CÔNG NGHỆ THÔNG TIN

PHIẾU ĐĂNG KÝ NHẬN ĐỒ ÁN TỐT NGHIỆP

1. Họ và tên sinh viên đăng ký đề tài

Họ và tên: Trương Ngự Quyền MSSV: 64131995 khóa: 64

Ngành: Công nghệ thông tin Khoa: Công nghệ thông tin

2. Tên đề tài đăng ký:

Nghiên cứu, cài đặt và cải tiến thuật toán phân cụm đỉnh mật độ dựa trên tối ưu tham số

3. Cán bộ hướng dẫn:

Họ và tên CBHD: TS. Nguyễn Thủy Đoan Trang

Sinh viên đã hiểu rõ yêu cầu của đề tài, cam kết thực hiện và hoàn thành theo đúng nội dung, thời hạn đề ra.

Khánh Hòa, ngày... tháng ... năm 2026

Ý kiến cán bộ hướng dẫn

(Ký và ghi rõ họ tên)

Sinh viên

(Ký và ghi rõ họ tên)

Trưởng Bộ môn duyệt

(Ký và ghi rõ họ tên)

PHIẾU THEO DÕI TIẾN ĐỘ VÀ ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP

(Dùng cho CBHD và nộp cùng báo cáo ĐA/KLTN của sinh viên)

Tên đề tài: Nghiên cứu, cài đặt và cải tiến thuật toán phân cụm đỉnh mật độ dựa trên tối ưu tham số

Chuyên ngành: Công nghệ thông tin

Họ và tên sinh viên: Trương Ngự Quyền

Mã sinh viên: 64131995

Người hướng dẫn (học hàm, học vị, họ và tên): TS. Nguyễn Thủy Đoan Trang

Cơ quan công tác: Khoa Công nghệ thông tin, Trường Đại học Nha Trang

Phần đánh giá và cho điểm của người hướng dẫn (tính theo thang điểm 10)

Tiêu chí đánh giá	Trọng số (%)	Mô tả mức chất lượng				Điểm
		Giỏi	Khá	Đạt yêu cầu	Không đạt	
		9 - 10	7 - 8	5 - 6	< 5	
Xây dựng đề cương nghiên cứu	10					
Tinh thần và thái độ làm việc	10					
Kiến thức và kỹ năng làm việc	10					
Nội dung và kết quả đạt được	40					
Kỹ năng viết và trình bày báo cáo	30					
ĐIỂM TỔNG						

Ghi chú: Điểm tổng làm tròn đến 1 số lẻ.

Nhận xét chung (sau khi sinh viên hoàn thành ĐA/KLTN):

.....
.....
.....
.....

Đồng ý cho sinh viên:

Được bảo vệ:

Không được bảo vệ:

Khánh Hòa, ngày...tháng...năm...

Cán bộ hướng dẫn

(Ký và ghi rõ họ tên)

II. Phần nhận xét cụ thể (dựa theo phiếu chấm điểm và khung tiêu chí đánh giá theo Rubric)

II.1. Hình thức thuyết minh (tỉ trọng 30%)

** **Trình bày** (Rõ ràng, mạch lạc? Biểu bảng, hình vẽ trình bày rõ ràng, đúng quy cách? ...)*

.....

** **Bố cục và lập luận** (Bố cục hợp lý? Tỉ trọng giữa các phần? Cơ sở lập luận? ...)*

.....

** **Văn phong** (Gọn gàng, súc tích hay rườm rà, khó hiểu? Lỗi văn phạm và chính tả? ...)*

.....

II.2. Nội dung thuyết minh (tỉ trọng 30%)

** **Mục tiêu nghiên cứu** (Trình bày rõ ràng? Ý nghĩa khoa học và thực tiễn? Tính khả thi? ...)*

.....

** **Tổng quan tài liệu** (Phân tích và đánh giá? Độ tin cậy và chất lượng nguồn tài liệu? ...)*

.....

** **Phương pháp nghiên cứu** (Hiện đại? Phù hợp với mục tiêu và nội dung nghiên cứu? Mô tả? Đánh giá và so sánh với các phương pháp khác? ...)*

.....

II.3. Kết quả nghiên cứu (tỉ trọng 20%)

** **Kết quả đạt được** (Độ tin cậy? Tính sáng tạo? Giá trị khoa học và thực tiễn? ...)*

.....

** **Kết luận** (Đáp ứng mục tiêu nghiên cứu? Quan điểm của cá nhân? ...)*

.....

4. MỨC ĐỘ TRÍCH DẪN VÀ SAO CHÉP (tỉ trọng 20%)

** **Mức độ trích dẫn** (Đúng quy định? Trung thực, đầy đủ, rõ ràng? Sắp xếp tài liệu tham khảo? ...)*

.....

** **Mức độ sao chép** (Tỉ lệ sao chép? Hình thức sao chép? ...)*

.....

LỜI CAM ĐOAN

Tôi xin cam đoan đồ án tốt nghiệp với đề tài “*Nghiên cứu, cài đặt và cải tiến thuật toán phân cụm đỉnh mật độ dựa trên tối ưu tham số*” là công trình nghiên cứu do bản thân tôi tự thực hiện dưới sự hướng dẫn khoa học của cô Nguyễn Thủy Đoan Trang.

Tôi cam kết toàn bộ kết quả thực nghiệm và số liệu trong đồ án là hoàn toàn trung thực, khách quan và chưa từng được công bố trong bất kỳ công trình nào khác. Mọi nội dung tham khảo, lý thuyết hay thuật toán kế thừa đều được trích dẫn nguồn gốc rõ ràng, minh bạch và đầy đủ trong danh mục tài liệu tham khảo theo quy định của Nhà trường.

Tôi xin hoàn toàn chịu trách nhiệm trước Hội đồng bảo vệ và Nhà trường về tính liêm chính học thuật của đồ án này. Nếu có bất kỳ gian lận nào hay sao chép trái quy định nào, tôi xin chịu mọi hình thức kỷ luật theo quy chế hiện hành.

LỜI CẢM ƠN

Để hoàn thành đồ án tốt nghiệp này, em xin chân thành cảm ơn quý Thầy Cô Khoa Công nghệ thông tin, Trường Đại học Nha Trang đã truyền đạt kiến thức và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập và thực hiện đề tài.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến cô Nguyễn Thủy Đoan Trang, người đã trực tiếp định hướng, tận tình hướng dẫn và động viên em trong suốt thời gian qua để đồ án được hoàn thành trọn vẹn.

Do năng lực cá nhân và thời gian nghiên cứu còn hạn chế, báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu của quý thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

TÓM TẮT

Thuật toán phân cụm mật độ CSDPPO giải quyết tốt việc tự động tối ưu tham số khoảng cách cắt d_c qua Entropy, nhưng bước xác định khoảng cách δ vẫn yêu cầu số lượng lớn phép so sánh khoảng cách khi dữ liệu tăng. Nhằm khắc phục hạn chế này, đề án đề xuất giải pháp cải tiến mang tên Pruned Grid Delta (PGD).

Phương pháp PGD thực hiện phân hoạch không gian dữ liệu thành các ô lưới (Grid) kích thước động. Quá trình tìm kiếm điểm có mật độ cao hơn để tính δ được triển khai mở rộng dần theo bán kính lưới, kết hợp áp dụng quy tắc cắt tỉa nghiêm ngặt dựa trên khoảng cách tối thiểu đến hộp bao (Bounding box) của từng ô. Cơ chế này cho phép loại bỏ sớm các vùng không gian không tiềm năng mà không cần truy xuất chi tiết các điểm bên trong, giúp giảm số lượng phép so sánh khoảng cách cần thực hiện.

Kết quả thực nghiệm chứng minh giải pháp PGD giúp tăng tốc độ xử lý so với CSDPPO gốc khi quy mô dữ liệu tăng lên. Đặc biệt, chiến lược cắt tỉa này dựa trên các cơ sở hình học chặt chẽ, do đó bảo toàn giá trị các tham số, giữ đúng cấu trúc đồ thị quyết định (Decision Graph).

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
TÓM TẮT	iii
MỤC LỤC	iv
DANH MỤC HÌNH VẼ, ĐỒ THỊ	vii
DANH MỤC BẢNG BIỂU	viii
DANH MỤC TỪ VIẾT TẮT	ix
PHẦN MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN	2
1.1. TỔNG QUAN VỀ PHÂN CỤM DỮ LIỆU	2
1.2. CÁC PHƯƠNG PHÁP PHÂN CỤM HIỆN NAY	2
1.2.1. Phương pháp phân cụm phân hoạch (Partitioning Methods)	2
1.2.2. Phương pháp phân cụm phân cấp	3
1.2.3. Phương pháp phân cụm dựa trên mật độ	5
1.3. ỨNG DỤNG CỦA PHÂN CỤM DỮ LIỆU	7
1.4. THÁCH THỨC TRONG BÀI TOÁN PHÂN CỤM DỮ LIỆU	7
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	9
2.1. THUẬT TOÁN PHÂN CỤM CFSFDP	9
2.1.1. Mật độ cục bộ ρ	9
2.1.2. Khoảng cách δ	9
2.1.3. Decision Graph	10
2.1.4. Các bước thực hiện thuật toán CFSFDP	11
2.1.5. Đánh giá thuật toán	14
2.2. THUẬT TOÁN PHÂN CỤM CSDPPO	14
2.2.1. Giới thiệu	14
2.2.2. Tính mật độ cục bộ bằng Gaussian Kernel	14
2.2.3. Hàm Entropy mật độ	15
2.2.4. Quy trình thực hiện thuật toán CSDPPO	16
2.2.5. Đánh giá thuật toán CSDPPO	19

2.3. ĐỊNH HƯỚNG NGHIÊN CỨU CỦA ĐỀ TÀI	19
2.4. ĐỘ ĐO KHOẢNG CÁCH	20
2.5. GRID VÀ BOUNDING BOX	20
CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT	22
3.1. HẠN CHẾ CỦA BƯỚC XÁC ĐỊNH δ TRONG CSDPPO	22
3.1.1. Quá trình xác định khoảng cách δ	22
3.1.2. Hạn chế về chi phí tính toán	22
3.2. PHƯƠNG PHÁP PGD ĐỀ XUẤT	23
3.2.1. Ý tưởng của phương pháp PGD	23
3.2.2. Cấu trúc dữ liệu grid	24
3.2.2.1. <i>Global Bounding Box</i>	25
3.2.2.2. <i>Tính toán kích thước lưới</i>	26
3.2.2.3. <i>Cơ chế đánh chỉ số ô lưới</i>	27
3.2.2.4. <i>Xác định một điểm thuộc một ô lưới (Point Mapping)</i>	28
3.2.2.5. <i>Xác định biên tọa độ của một ô lưới</i>	29
3.2.3. Pruning theo Bounding Box	30
3.2.3.1. <i>Vai trò của biên ô trong cơ chế cắt tỉa</i>	31
3.2.3.2. <i>Chiến lược mở rộng không gian tìm kiếm theo bán kính đồng tâm</i>	32
3.2.3.3. <i>Thứ tự duyệt chi tiết</i>	32
3.2.4. Phương pháp PGD	34
3.2.5. Ví dụ minh họa quá trình xác định δ bằng phương pháp PGD	37
3.2.6. Phân tích độ phức tạp	41
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ	43
4.1. MÔI TRƯỜNG THỰC NGHIỆM	43
4.2. DỮ LIỆU THỰC NGHIỆM	43
4.3. TIÊU CHÍ ĐÁNH GIÁ	44
4.3.1. Thời gian xử lý	44
4.3.2. Số lượng phép so sánh khoảng cách	45
4.3.3. Độ chính xác phân cụm	45
4.4. KẾT QUẢ THỰC NGHIỆM	46

4.4.1. So sánh thời gian xử lý	47
4.4.2. So sánh số lượng phép so sánh khoảng cách	49
4.4.3. So sánh kết quả phân cụm	50
4.4.4. So sánh PGD-CSDPPO với giải thuật SDPC (2024)	51
4.5. Thảo luận kết quả	52
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	54
5.1. KẾT LUẬN	54
5.1.1. Các đóng góp chính của đề tài	54
5.1.2. Hạn chế của đề tài	55
5.2. HƯỚNG PHÁT TRIỂN	55
TÀI LIỆU THAM KHẢO	57

DANH MỤC HÌNH VẼ, ĐỒ THỊ

Hình 1.1. Minh họa thuật toán K-means	3
Hình 1.2. Minh họa phương pháp phân cụm phân cấp	4
Hình 1.3. Kết quả phân cụm giữa K-means và phương pháp dựa trên mật độ	5
Hình 2.1. Minh họa Decision Graph	11
Hình 2.2. Minh họa hộp bao (Bounding Box)	21
Hình 3.1. Minh họa dòng thác mật độ giảm dần	23
Hình 3.2. Minh họa cấu trúc không gian lưới	24
Hình 3.3. Sơ đồ minh họa khung giới hạn toàn cục	26
Hình 3.4. Sơ đồ cơ chế đánh chỉ số ô lưới (Grid Indexing)	28
Hình 3.5. Sơ đồ hình học xác định các đường biên tạo độ ($x1, x2, y1, y2$) của một ô lưới từ chỉ số (col, row)	29
Hình 3.6. Minh họa vòng tròn cắt tỉa hình học	31
Hình 3.7. Minh họa khoảng cách từ điểm truy vấn đến Bounding Box	32
Hình 3.8. Minh họa cơ chế mở rộng không gian tìm kiếm theo các lớp vỏ bán kính đồng tâm (r)	32
Hình 3.9. Minh họa thứ tự duyệt các ô	34
Hình 3.10. Minh họa lưu đồ của phương pháp PGD	35
Hình 3.11. Minh họa các điểm trên ô lưới	38
Hình 4.1. Kết quả phân cụm bộ dữ liệu Flame	46
Hình 4.2. Kết quả phân cụm bộ dữ liệu Spiral	46
Hình 4.3. Kết quả phân cụm bộ dữ liệu Aggregation	47
Hình 4.4. Đồ thị so sánh thời gian chạy – Runtime	48
Hình 4.5. Đồ thị so sánh số phép so sánh khoảng cách	49

DANH MỤC BẢNG BIỂU

Bảng 2.1. So sánh CFSFDP và CSDPPO	18
Bảng 3.1. Thứ tự duyệt chi tiết của 8 ô xung quanh	33
Bảng 3.2. Dữ liệu minh họa cho phương pháp PGD	37
Bảng 4.1. Tóm tắt các bộ dữ liệu	44
Bảng 4.2. Kết quả thực nghiệm các bộ dữ liệu thành phố và dữ liệu nhóm	47
Bảng 4.3: Bảng so sánh thời gian thực thi bước tính δ giữa hai thuật toán	48
Bảng 4.4: Thống kê số lượng phép so sánh khoảng cách và tỷ lệ cắt tỉa	49
Bảng 4.5: Đối chiếu chất lượng phân cụm (ARI và NMI) giữa hai thuật toán	50

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Tiếng Việt
ARI	Adjusted Rand Index	Chỉ số Rand điều chỉnh
CFSFDP	Clustering by Fast Search and Find of Density Peaks	Thuật toán CFSFDP
CSDPPO	Clustering by Searching Density Peaks based on Parameter Optimization	Thuật toán CSDPPO
DBSCAN	Density-Based Spatial Clustering	Phân cụm không gian dựa trên mật độ
NMI	Normalized Mutual Information	Độ đo thông tin tương hỗ chuẩn hóa
OPTICS	Ordering Points To Identify the Clustering Structure	Thuật toán sắp xếp điểm nhằm phát hiện cấu trúc cụm dữ liệu
PGD	Pruned Grid Delta	Phương pháp PGD
SDPC	Sequential Density Peak Clustering	Thuật toán SDPC

PHẦN MỞ ĐẦU

Trong lĩnh vực khai phá dữ liệu lớn, phân cụm dữ liệu là một kỹ thuật cốt lõi giúp tự động phát hiện cấu trúc ngầm và phân tách các nhóm thông tin có mật độ tương đồng cao. Trong đó, các phương pháp phân cụm dựa trên mật độ thể hiện ưu thế vượt trội nhờ khả năng tự động nhận diện hình dạng phức tạp và loại bỏ các phần tử nhiễu hiệu quả. Tuy nhiên, khi quy mô dữ liệu tăng lên, hướng tiếp cận này thường gặp vấn đề nghiêm trọng về mặt hiệu năng ở bước tìm kiếm và so sánh khoảng cách, dẫn đến chi phí tính toán tăng theo hàm bậc hai. Việc phải lặp đi lặp lại quá nhiều phép tính trùng lặp trên toàn bộ không gian dữ liệu đã làm giảm tính khả thi khi triển khai mô hình trong thực tế.

Nhằm khắc phục hạn chế này, đề tài tập trung nghiên cứu giải pháp phân hoạch không gian và cắt tỉa thông minh để tối ưu hóa hiệu năng xử lý. Nghiên cứu hướng tới mục tiêu giảm thiểu thời gian tính toán nhưng vẫn bảo toàn độ chính xác của kết quả. Trọng tâm nội dung xoay quanh cấu trúc dữ liệu lưới và các chiến thuật cắt tỉa không gian dựa trên các giới hạn hình học của vùng.

CHƯƠNG 1. TỔNG QUAN

1.1. TỔNG QUAN VỀ PHÂN CỤM DỮ LIỆU

Hiện nay, với sự phát triển của công nghệ thông tin và Internet, lượng dữ liệu được tạo ra ngày càng nhiều và đa dạng. Dữ liệu xuất hiện trong nhiều lĩnh vực như giáo dục, y tế, thương mại điện tử,... Tuy nhiên, dữ liệu sau khi thu thập thường tồn tại dưới dạng thô, chưa được phân tích và rất khó khai thác nếu không có các phương pháp xử lý phù hợp. Do đó, lĩnh vực khai phá dữ liệu được nghiên cứu nhằm hỗ trợ quá trình tìm kiếm thông tin và tri thức có ý nghĩa từ dữ liệu [1], [2].

Phân cụm dữ liệu (Data Clustering) là một trong những kỹ thuật nền tảng và quan trọng nhất trong học máy không giám sát (Unsupervised Learning) và khai phá dữ liệu (Data Mining) [3], [4].

Khác với các bài toán trong học máy giám sát, luôn đòi hỏi dữ liệu phải được gán nhãn trước, phân cụm dữ liệu hướng tới việc phân tích tập dữ liệu ban đầu dựa trên đặc trưng về hình học hoặc cấu trúc dữ liệu của các đối tượng [3].

Về mặt toán học, phân cụm dữ liệu hay gọi tắt là phân cụm là chia tập hợp n phần tử dữ liệu thành các nhóm riêng biệt. Sao cho ở mỗi nhóm, các phần tử bên trong có sự tương đồng và ngược lại, các phần tử thuộc các nhóm khác nhau có sự khác biệt [3], [5].

Phân cụm dữ liệu hiện nay được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau của đời sống và công nghệ. Trong thương mại điện tử, kỹ thuật phân cụm được sử dụng để phân tích hành vi khách hàng, từ đó hỗ trợ xây dựng hệ thống gợi ý sản phẩm phù hợp. Trong lĩnh vực y tế, phân cụm giúp hỗ trợ phân tích dữ liệu bệnh án, nhận diện nhóm bệnh nhân có đặc điểm tương đồng nhằm phục vụ công tác chẩn đoán và điều trị. Ngoài ra, phân cụm còn được ứng dụng trong xử lý ảnh, phân tích văn bản, phát hiện bất thường trong mạng máy tính và nhiều bài toán khai phá dữ liệu khác [5].

Bên cạnh những ưu điểm về khả năng khám phá cấu trúc tiềm ẩn của dữ liệu, bài toán phân cụm cũng tồn tại nhiều thách thức như dữ liệu có kích thước lớn, phân bố phức tạp, mật độ không đồng đều hoặc chứa nhiễu. Do đó, nhiều phương pháp phân cụm khác nhau đã được nghiên cứu nhằm nâng cao độ chính xác và hiệu quả xử lý dữ liệu trong thực tế.

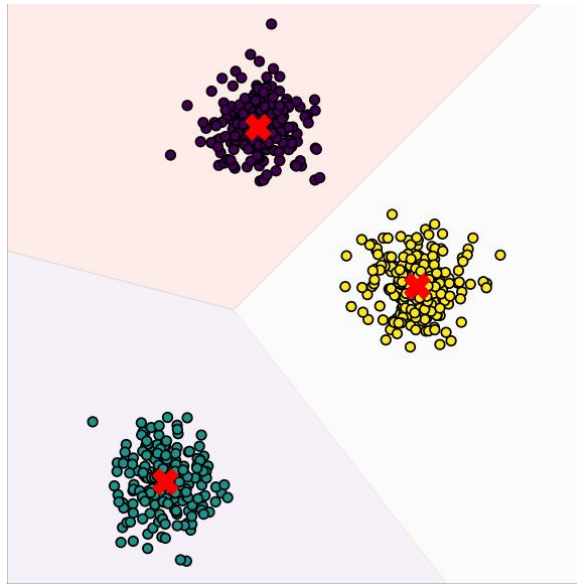
1.2. CÁC PHƯƠNG PHÁP PHÂN CỤM HIỆN NAY

1.2.1. Phương pháp phân cụm phân hoạch (Partitioning Methods)

Phương pháp phân cụm phân hoạch (Partitioning Clustering) là loại phân cụm chia trực tiếp tập dữ liệu gồm N đối tượng thành K cụm riêng biệt theo tham số chỉ định trước của người sử dụng. Đảm bảo mỗi phần tử thuộc về duy nhất

một cụm [3]. Thuật toán đại diện tiêu biểu và phổ biến cho hướng tiếp cận này là thuật toán K-means [3].

Quy trình phân chia dữ liệu của K-means dựa trên nguyên lý tối ưu hóa khoảng cách từ các điểm dữ liệu đến tâm cụm tương ứng. Cơ chế này được mô tả trực quan trong Hình 1.1 thông qua hình ảnh một không gian hình học tương tự như vùng biển chứa ba quần đảo lớn, một điểm dữ liệu sẽ được quy hoạch vào một quần đảo nếu khoảng cách từ nó đến tâm của đảo đó là ngắn nhất so với hai đảo còn lại [4].



Hình 1.1. Minh họa thuật toán K-means

Về mặt hiệu năng, phương pháp phân hoạch sở hữu ưu thế lớn về tốc độ xử lý nhờ cơ chế tính toán trực tiếp và lặp với số bước hữu hạn. Độ phức tạp thời gian đạt mức tuyến tính $O(tKN)$ (với t là số vòng lặp), giúp giải thuật vận hành nhanh chóng và đạt hiệu quả cao trên các tập dữ liệu có quy mô lớn [2], [6]. Dựa vào kết quả thực nghiệm trên Hình 1.1, có thể thấy K-means đạt hiệu quả tối ưu khi cấu trúc dữ liệu tường minh, tách biệt rõ ràng và có dạng phân phối cầu lồi đồng nhất.

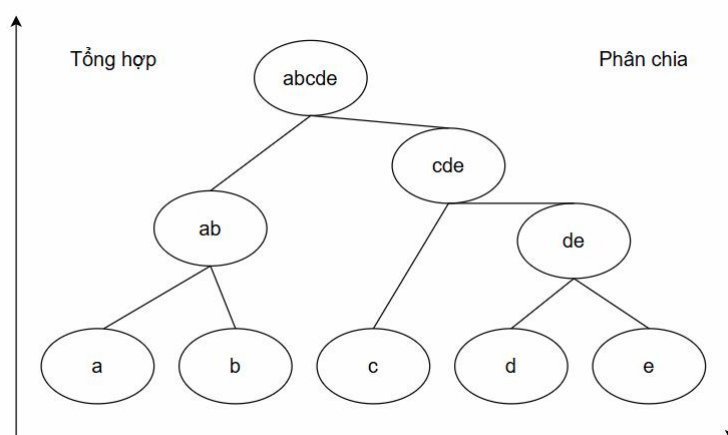
Tuy nhiên, phương pháp phân hoạch vẫn tồn tại hai hạn chế chính. Thứ nhất, thuật toán bắt buộc người dùng phải xác định trước số lượng cụm K . Việc thiết lập sai giá trị tham số K so với cấu trúc phân bố thực tế của dữ liệu sẽ làm ảnh hưởng kết quả, gây cường ép phân chia, làm sai lệch bản chất của thông tin. Thứ hai, thuật toán kém linh hoạt với các dữ liệu có hình dạng phức tạp. K-means chỉ nhận diện tốt các khối dữ liệu dạng cầu. Khi thay đổi sang các kiểu dữ liệu khác như đường xoắn ốc, hình bán nguyệt hoặc vòng cung lồng nhau, thuật toán khó có thể phân tách chính xác [5].

1.2.2. Phương pháp phân cụm phân cấp

Phương pháp phân cụm phân cấp (Hierarchical Clustering) xây dựng cấu trúc dữ liệu theo dạng mô hình phân tầng, và thường được biểu diễn trực quan dưới dạng đồ thị cây (Dendrogram). Khác với phương pháp phân cụm phân hoạch, hướng tiếp cận này không đòi hỏi phải xác định số lượng cụm K ngay từ giai đoạn khởi tạo. Dựa trên cơ chế vận hành, phương pháp này có thể được chia làm hai trường phái độc lập bao gồm: Cách tiếp cận tổng hợp (agglomerative) và cách tiếp cận phân chia (divisive).

- Đối với cách tiếp cận tổng hợp, có thể gọi là cách tiếp cận từ dưới lên. Khi bắt đầu, mỗi đối tượng sẽ nằm trong một nhóm riêng biệt. Và sau đó thuật toán sẽ liên tục hợp nhất các phần tử, hay nhóm có khoảng cách gần nhau nhất. Điều này được duy trì cho tới khi toàn bộ các nhóm nhỏ được gom lại thành một cụm duy nhất ở tầng cao nhất, hoặc dừng khi đạt đến điều kiện dừng [3], [6].
- Còn với cách tiếp cận phân chia, phương pháp này sẽ triển khai theo nguyên lý từ trên xuống. Khác với cách tiếp cận tổng hợp, khi bắt đầu các đối tượng dữ liệu được sắp xếp vào chung một cụm lớn duy nhất. Trong mỗi vòng lặp sau đó, một cụm liên tục bị chia thành các cụm con nhỏ hơn. Quá trình này diễn ra tuần tự và sẽ tiếp diễn cho tới khi mỗi đối tượng là một cụm độc lập, hoặc thuật toán thỏa mãn điều kiện kết thúc [3], [6].

Cấu trúc phân tầng dạng đồ thị cây của hai cách tiếp cận này được mô tả chi tiết trong Hình 1.2:



Hình 1.2. Minh họa phương pháp phân cụm phân cấp

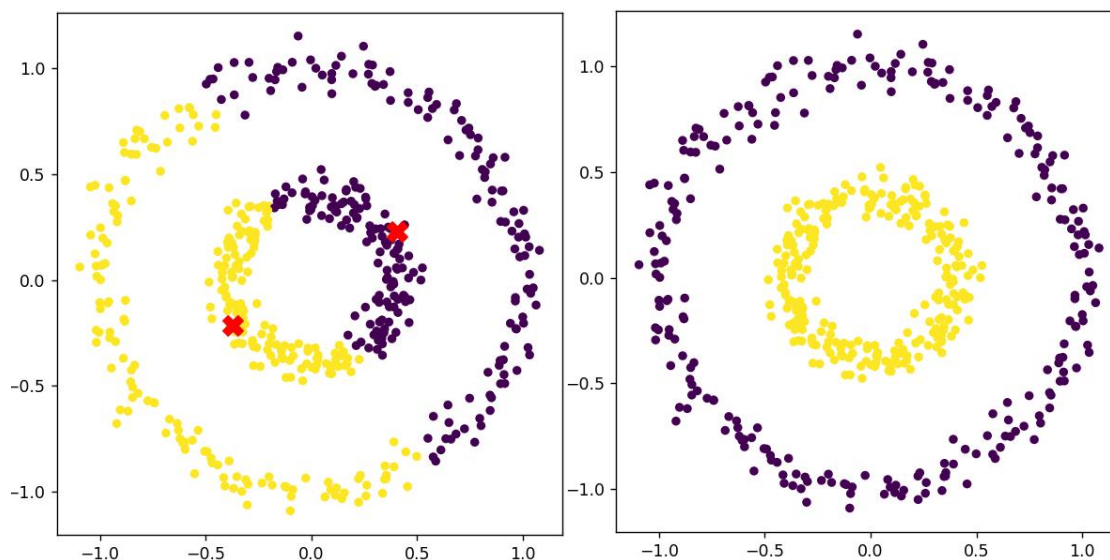
Phân tích từ sơ đồ Hình 1.2 cho thấy, các quyết định hợp nhất hoặc phân chia trong mô hình phân cấp có tính chất một chiều và không thể đảo ngược. Điều này đồng nghĩa với việc nếu một bước phân hoạch hoặc gom cụm ở giai đoạn trước bị sai lệch, thuật toán không có cơ chế sửa sai ở các bước tiếp theo.

Tóm lại, phương pháp phân cụm phân cấp sở hữu ưu thế lớn nhờ khả năng linh hoạt không phụ thuộc vào tham số cụm trước. Tuy nhiên, rào cản lớn nhất

của mô hình này lại là chi phí tính toán ma trận khoảng cách lớn và các quyết định hợp nhất hoặc phân chia không thể đảo ngược.

1.2.3. Phương pháp phân cụm dựa trên mật độ

Phần lớn các phương pháp phân cụm truyền thống hoạt động dựa trên khoảng cách giữa các đối tượng dữ liệu. Và phân cụm dựa trên các dạng dữ liệu có hình dạng cầu, với các dạng dữ liệu khác như hình bán nguyệt, hình xoắn ốc, bộ dữ liệu có nhiễu,... có thể gặp không ít khó khăn. Hình 1.3 minh họa sự khác biệt về năng lực nhận diện cấu trúc hình học phức tạp giữa K-means và các phương pháp dựa trên mật độ.



Hình 1.3. Kết quả phân cụm giữa K-means và phương pháp dựa trên mật độ

Kết quả trực quan từ Hình 1.3 cho thấy cách tiếp cận dựa trên mật độ có ưu thế vượt trội trong việc bao bọc và nhận diện các hình dạng cụm bất kỳ.

Điểm đặc biệt của phương pháp này là khả năng phát hiện số lượng cụm phù hợp dựa trên phân bố dữ liệu, và có thể loại bỏ các điểm nhiễu một cách hiệu quả. Đại diện cho phương pháp này có thể kể đến các thuật toán như DBSCAN [7], OPTICS [8].

Một bước tiến quan trọng khác là thuật toán CFSFDP (Clustering by Fast Search and Find of Density Peaks) được đề xuất bởi hai tác giả Alex Rodriguez và Alessandro Laio vào năm 2014 [9], dựa trên hai giả định rằng: các tâm cụm luôn có mật độ cục bộ cao hơn các điểm xung quanh và nằm cách xa các điểm có mật độ cao hơn khác.

Mặc dù sở hữu nhiều ưu điểm vượt trội, bản thân các phương pháp này cũng tồn tại những nhược điểm nhất định, đặc điểm chung là rất nhạy với các tham số đầu vào. Nếu dữ liệu có sự phân bố không đồng đều, một giá trị tham số cố định có thể làm ảnh hưởng đến khả năng nhận diện chính xác cấu trúc cụm và khiến tâm cụm bị lệch so với thực tế.

Nhằm khắc phục một phần các hạn chế của các phương pháp phân cụm dựa trên mật độ truyền thống, nhiều hướng tiếp cận mới đã được nghiên cứu và phát triển. Trong đó, thuật toán CFSFDP được đánh giá là một trong những phương pháp nổi bật nhờ khả năng xác định tâm cụm dựa trên đặc trưng mật độ cục bộ và khoảng cách tương đối giữa các điểm dữ liệu.

1.3. ỨNG DỤNG CỦA PHÂN CỤM DỮ LIỆU

Cùng với sự bùng nổ của dữ liệu lớn (Big Data), phân cụm dữ liệu (Data Clustering) đã khẳng định vị thế là một trong những kỹ thuật cốt lõi nhất thuộc học máy không giám sát (Unsupervised Learning). Phương pháp này tự động phân loại các đối tượng vào các nhóm riêng biệt dựa trên nguyên lý tối đa hóa độ tương đồng trong cùng cụm và tối đa hóa độ khác biệt giữa các cụm [3]. Do vận hành không phụ thuộc vào dữ liệu gán nhãn, một nguồn tài nguyên khan hiếm và tốn kém, phân cụm dữ liệu đóng vai trò then chốt trong việc khai phá tri thức và tối ưu hóa các quyết định chiến lược thực tiễn [3].

Trong thương mại điện tử và kinh doanh số, các thuật toán tiêu biểu như K-Means [10] hay DBSCAN [7] giúp phân khúc khách hàng tự động dựa trên lịch sử giao dịch và hành vi tương tác. Kết quả này hỗ trợ đắc lực cho các hệ thống gợi ý nhằm cá nhân hóa trải nghiệm người dùng, đồng thời giúp doanh nghiệp dự báo nguy cơ rời bỏ dịch vụ để kịp thời tối ưu chuỗi cung ứng [5].

Đối với y tế thông minh, kỹ thuật này giúp phân nhóm bệnh nhân từ hồ sơ bệnh án điện tử (EHR) theo tổ hợp triệu chứng và chỉ số hóa, hỗ trợ bác sĩ phân tầng nguy cơ và cá nhân hóa phác đồ điều trị. Bên cạnh đó, trong xử lý ảnh y khoa (CT, MRI), phân cụm các điểm ảnh tương đồng về mức xám hoặc kết cấu cho phép hệ thống tự động phân đoạn cấu trúc giải phẫu và định vị chính xác tổn thương [11], giảm thiểu sai sót chủ quan trong chẩn đoán.

Trong an ninh mạng, phân cụm là giải pháp đột phá để phát hiện hành vi bất thường. Cơ chế này giả định lưu lượng mạng hợp lệ sẽ tập trung thành các cụm mật độ cao, trong khi các hành vi tấn công sẽ phân bố tách biệt [12]. Bằng cách cô lập các điểm dữ liệu dị biệt này, hệ thống có thể chủ động ngăn chặn các mối đe dọa theo thời gian thực, ngay cả với các lỗ hổng bảo mật mới [12].

Trong xử lý ảnh và thị giác máy tính, thuật toán nhóm các điểm ảnh (pixels) có đặc trưng tương đồng về màu sắc hoặc kết cấu để phục vụ bài toán phân đoạn ảnh (Image Segmentation) [13]. Quy trình này đơn giản hóa hình ảnh phức tạp để tối ưu hóa việc tách nền, nhận dạng vật thể và nén dữ liệu, làm tiền đề cho các ứng dụng nâng cao như nhận diện khuôn mặt hay xe tự hành.

Ngoài các lĩnh vực trên, phân cụm dữ liệu còn chứng minh tính vạn năng trong việc phát hiện cộng đồng mạng xã hội, giám sát dữ liệu cảm biến IoT, phân loại chủ đề văn bản [11] và phân tích biểu hiện gene trong tin sinh học. Với khả năng thích nghi linh hoạt trên mọi định dạng dữ liệu mà không cần sự can thiệp của con người, phân cụm tiếp tục giữ vững vị thế là mũi nhọn nghiên cứu quan trọng trong lĩnh vực trí tuệ nhân tạo toàn cầu [6].

1.4. THÁCH THỨC TRONG BÀI TOÁN PHÂN CỤM DỮ LIỆU

Mặc dù đạt được nhiều thành tựu trong cả nghiên cứu lẫn thực tiễn, bài toán phân cụm dữ liệu vẫn đối mặt với nhiều thách thức lớn về đặc điểm dữ liệu và

hiệu năng tính toán. Khó khăn phổ biến nhất là việc xác định số lượng cụm phù hợp. Một số thuật toán như K-Means yêu cầu người dùng cấu hình trước tham số K [5], [10], song trong thực tế, do cấu trúc dữ liệu ban đầu thường không tường minh, việc lựa chọn sai số lượng cụm sẽ làm giảm đáng kể chất lượng phân cụm. Bên cạnh đó, dữ liệu thực tế thường phân bố dưới nhiều hình dạng phức tạp và không đồng đều. Các phương pháp truyền thống dựa trên khoảng cách chỉ hoạt động hiệu quả với các cụm dạng cầu hoặc phân bố đồng nhất; ngược lại, chúng thường thất bại trước các tập dữ liệu có dạng hình học đặc thù như xoắn ốc, bán nguyệt, cụm lồng nhau hoặc có mật độ không đồng đều [3], [5]. Ngoài ra, nhiễu và các điểm ngoại lai cũng ảnh hưởng tiêu cực đến quá trình phân cụm khi dễ dàng làm sai lệch vị trí tâm cụm hoặc biến đổi cấu trúc phân nhóm nếu thuật toán thiếu cơ chế xử lý phù hợp.

Đối với các phương pháp phân cụm dựa trên mật độ như CFSFDP [9] hay CSDPPO [14], bài toán tối ưu hóa tham số đóng vai trò quyết định. Chất lượng phân cụm phụ thuộc chặt chẽ vào các chỉ số đầu vào, điển hình là khoảng cách cắt d_c . Nếu tham số này không tương thích với đặc điểm phân bố dữ liệu, quy trình tính toán mật độ cục bộ và xác định tâm cụm sẽ bị sai lệch. Song song đó, chi phí tính toán là một rào cản lớn khi xử lý các tập dữ liệu quy mô lớn. Nhiều thuật toán dựa trên mật độ yêu cầu tính toán khoảng cách giữa mọi cặp điểm dữ liệu, dẫn đến độ phức tạp thuật toán cao. Khi số lượng phần tử tăng lên, thời gian xử lý và số lượng phép so sánh khoảng cách sẽ tăng theo cấp số nhân, gây quá tải hệ thống.

Những hạn chế kể trên đòi hỏi việc phát triển các phương pháp phân cụm mới có khả năng thích nghi tốt với cấu trúc dữ liệu phức tạp, giảm thiểu sự phụ thuộc vào tham số cấu hình và tối ưu hóa chi phí tính toán. Đây cũng chính là động lực để đề tài tập trung nghiên cứu, cải tiến bước xác định khoảng cách δ trong thuật toán CSDPPO nhằm nâng cao hiệu năng xử lý trên các tập dữ liệu lớn.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. THUẬT TOÁN PHÂN CỤM CFSFDP

Thuật toán CFSFDP (Clustering by Fast Search and Find of Density Peaks) hoạt động dựa trên giả định rằng: các tâm cụm là điểm dữ liệu có mật độ cao và cao hơn các điểm xung quanh, và các tâm cụm luôn cách xa các điểm khác có mật độ cao hơn [9].

Từ tư duy này, các tâm cụm tiềm năng được thuật toán xác định dựa trên hai tham số bao gồm mật độ cục bộ mỗi điểm ρ và khoảng cách gần nhất đến điểm có mật độ cao hơn δ .

2.1.1. Mật độ cục bộ ρ

Tham số đầu tiên ρ đại diện cho mức độ dày đặc của các phần tử xung quanh nó trong một phạm vi cố định cho trước. Giá trị này được xác định thông qua hàm Cut-off kernel dựa trên khoảng cách cắt d_c [9], hoặc có thể xem như bán kính bao phủ do người dùng tự xác định.

Công thức tính mật độ cho mỗi điểm:

$$\rho_i = \sum_{j \neq i} \chi(d_{ij} - d_c) \quad (2.1)$$

Trong đó, d_{ij} là khoảng cách Euclidean giữa 2 điểm i và j . Hàm $\chi(x)$ được định nghĩa như sau: nhận giá trị bằng 1 nếu $x < 0$ và bằng 0 trong các trường hợp ngược lại:

$$\chi(x) = \begin{cases} 1 & \text{nếu } x < 0 \\ 0 & \text{ngược lại} \end{cases} \quad (2.2)$$

Công thức này thể hiện rằng tổng các điểm nằm trong vùng lân cận bán kính d_c chính là mật độ của điểm đó.

Hoặc công thức thứ hai là Gaussian kernel, ngoài Cut-off kernel bên trên, tác giả cũng đề cập đến việc sử dụng Gaussian kernel trong một số trường hợp:

$$\rho_i = \sum_{j \neq i} e^{-(d_{ij}/d_c)^2} \quad (2.3)$$

Sự khác biệt giữa hai công thức nằm ở cơ chế xác định mức độ ảnh hưởng của các điểm lân cận, ở công thức Cut-off biểu thị sự cắt cứng (hard thresholding), tức là mật độ mỗi điểm dựa vào ảnh hưởng của các điểm nằm trong bán kính d_c . Còn công thức thứ hai Gaussian biểu thị cắt mềm (soft thresholding), các điểm dù xa hay gần cũng đều ảnh hưởng ít nhiều đến mật độ của điểm đó.

2.1.2. Khoảng cách δ

Tham số thứ hai là δ đại diện cho mức độ tách biệt trong không gian dữ liệu giữa một điểm với các vùng có mật độ dày đặc hơn. Giá trị này được xác định bằng khoảng cách ngắn nhất từ điểm đang xét đến bất kỳ điểm nào khác mà có mật độ cao hơn [9], cụ thể công thức tính như sau:

$$\delta_i = \min_{j: \rho_j > \rho_i} (d_{ij}) \quad (2.4)$$

Riêng đối với phần tử có mật độ cực bộ lớn nhất trên toàn bộ tập dữ liệu. Vì không có điểm nào có mật độ cao hơn nó, giá trị δ của nó được tính bằng khoảng cách lớn nhất giữa nó với bất kỳ điểm nào trong không gian dữ liệu.

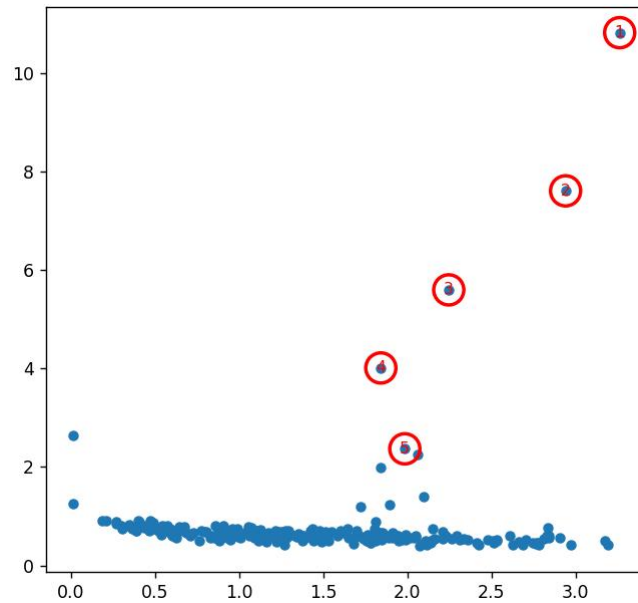
$$\delta_i = \max_j (d_{ij}) \quad (2.5)$$

2.1.3. Decision Graph

Trong biểu đồ Decision Graph, trục hoành biểu diễn giá trị mật độ cực bộ ρ , còn trục tung biểu diễn giá trị khoảng cách δ . Theo giả định của thuật toán CFSFDP, một tâm cụm lý tưởng phải đồng thời thỏa mãn hai điều kiện: nằm trong vùng có mật độ dữ liệu cao và cách xa các điểm có mật độ cao hơn khác. Do đó, các tâm cụm thường xuất hiện tách biệt tại khu vực phía trên bên phải của đồ thị.

Sau khi hoàn thành việc tính toán cặp tham số (ρ, δ) cho tất cả các phần tử, thuật toán tiến hành biểu diễn chúng trên một không gian hai chiều gọi là biểu đồ hay đồ thị quyết định (Decision Graph). Các điểm có tiềm năng trở thành tâm cụm tự nhiên sẽ xuất hiện ở góc trên bên phải của đồ thị, tương ứng với điểm vừa có mật độ cực bộ ρ cao mà vừa có khoảng cách δ lớn [9]. Ngược lại, một số điểm nhiều thường có giá trị ρ thấp nhưng δ tương đối lớn. Cơ chế này giúp CFSFDP nhận diện tâm cụm một cách trực quan mà không cần thực hiện các vòng lặp cập nhật liên tục để xác định tâm như thuật toán K-means [9].

Hình 2.1 minh họa về biểu đồ quyết định (Decision Graph) của bộ dữ liệu flame, các điểm nằm ở góc trên bên phải là các tâm cụm tiềm năng, trong khi các điểm nằm bên trái có mật độ thấp nhưng nhô cao biểu thị các điểm nhiễu.



Hình 2.1. Minh họa Decision Graph

2.1.4. Các bước thực hiện thuật toán CFSFDP

Thuật toán CFSFDP được thực hiện thông qua các bước chính sau:

Bước 1: Tính ma trận khoảng cách

- Đầu tiên, thuật toán tiến hành tính khoảng cách giữa mọi cặp điểm dữ liệu trong tập dữ liệu đầu vào. Thông thường, khoảng cách Euclidean được sử dụng để đo độ tương đồng giữa các điểm.
- Kết quả của bước này là một ma trận khoảng cách $D = [d_{ij}]$, trong đó mỗi phần tử d_{ij} biểu diễn khoảng cách giữa điểm i và điểm j .

Bước 2: Xác định mật độ cục bộ ρ

- Sau khi có ma trận khoảng cách, thuật toán tiến hành tính mật độ cục bộ ρ_i cho từng điểm dữ liệu dựa trên khoảng cách cắt d_c .
- Với mỗi điểm i , thuật toán sẽ kiểm tra các điểm còn lại:
 - Nếu khoảng cách giữa hai điểm nhỏ hơn d_c , điểm đó được xem là thuộc vùng lân cận của i ;
 - Ngược lại, điểm đó sẽ không đóng góp vào mật độ của điểm i .
- Giá trị ρ_i càng lớn cho thấy điểm dữ liệu đang xét nằm trong khu vực có mật độ dữ liệu cao.

Bước 3: Tính khoảng cách δ

- Sau khi xác định mật độ cục bộ, thuật toán tiếp tục tính giá trị δ_i cho từng điểm.
- Đối với mỗi điểm dữ liệu:
 - Thuật toán tìm tất cả các điểm có mật độ lớn hơn điểm đang xét;
 - Sau đó xác định khoảng cách nhỏ nhất từ điểm hiện tại đến các điểm có mật độ cao hơn đó.

- Riêng đối với điểm có mật độ lớn nhất trong toàn bộ tập dữ liệu:
 - Vì không tồn tại điểm nào có mật độ cao hơn;
 - Giá trị δ của điểm này được gán bằng khoảng cách lớn nhất từ nó đến các điểm còn lại.

Bước 4: Xây dựng biểu đồ Decision Graph

- Sau khi tính toán hoàn tất hai tham số (ρ, δ) , thuật toán biểu diễn tất cả các điểm dữ liệu trên biểu đồ hai chiều gọi là Decision Graph.
- Trong biểu đồ:
 - Trục hoành biểu diễn giá trị mật độ cục bộ ρ ;
 - Trục tung biểu diễn giá trị khoảng cách δ .
- Thông qua biểu đồ này:
 - Các điểm có ρ cao và δ lớn thường xuất hiện nổi bật ở góc trên bên phải;
 - Đây chính là các tâm cụm tiềm năng của dữ liệu.

Bước 5: Xác định tâm cụm

- Dựa trên Decision Graph, thuật toán tiến hành lựa chọn các điểm có giá trị ρ cao và δ lớn làm tâm cụm.
- Việc lựa chọn này có thể:
 - Thực hiện trực quan thông qua quan sát biểu đồ;
 - Hoặc sử dụng thêm các ngưỡng điều kiện để tự động xác định.
- Các điểm được chọn sẽ đóng vai trò đại diện cho từng cụm dữ liệu.

Bước 6: Gán nhãn cụm cho các điểm còn lại

- Sau khi xác định các tâm cụm, thuật toán tiến hành gán nhãn cho toàn bộ các điểm dữ liệu còn lại.
- Mỗi điểm sẽ được gán vào cùng cụm với điểm gần nhất có mật độ cao hơn nó.
- Quá trình này được thực hiện lặp lại cho đến khi tất cả các điểm dữ liệu đều được phân vào một cụm xác định.

Mã giả thuật toán CFSFDP được trình bày như sau:

Algorithm 1: CFSFDP Clustering Algorithm

Input:

$D = \{x_1, x_2, \dots, x_n\}$: tập dữ liệu

d_c : khoảng cách cắt

Output:

C : tập các cụm dữ liệu

- (1) Compute distance matrix d_{ij} for all data points
 - (2) for each point $x_i \in D$ do
 - (3) Compute local density ρ_i
 - (4) end for
 - (5) for each point $x_i \in D$ do
 - (6) Find the nearest point having higher density
 - (7) Compute δ_i
 - (8) end for
 - (9) Construct Decision Graph using (ρ_i, δ_i)
 - (10) Select cluster centers based on large values of ρ and δ
 - (11) for each non-center point x_i do
 - (12) Assign x_i to the same cluster as its nearest higher-density neighbor
 - (13) end for
 - (14) Return clustering result C
-

2.1.5. Đánh giá thuật toán

Ưu điểm của CFSFDP là có khả năng phát hiện các cụm có hình dạng phức tạp mà không cần biết trước hình dạng dữ liệu ban đầu. Thuật toán cũng không yêu cầu xác định trước số lượng cụm như phương pháp K-means.

Mặc dù vậy, CFSFDP vẫn tồn tại một số hạn chế. Thuật toán phụ thuộc mạnh vào tham số khoảng cách cắt d_c . Nếu giá trị d_c không phù hợp, kết quả phân cụm có thể bị ảnh hưởng đáng kể. Bên cạnh đó, bước tính khoảng cách δ cũng cần phải so sánh khoảng cách giữa các điểm dữ liệu với nhau, dẫn đến chi phí tính toán lớn khi số lượng dữ liệu tăng cao. Đây cũng là một trong những nguyên nhân làm giảm hiệu năng của thuật toán trên các tập dữ liệu lớn.

2.2. THUẬT TOÁN PHÂN CỤM CSDPPO

2.2.1. Giới thiệu

Như đã phân tích ở mục trước, mặc dù thuật toán CFSFDP cho kết quả phân cụm tốt trên nhiều bộ dữ liệu khác nhau, nhưng phương pháp này vẫn tồn tại nhiều mặt hạn chế. Một trong các hạn chế đó là sự phụ thuộc lớn vào tham số khoảng cách cắt (d_c). Việc lựa chọn sai giá trị có thể làm ảnh hưởng đến mật độ các điểm, gián tiếp ảnh hưởng đến biểu diễn của các điểm trên Decision Graph, làm giảm chất lượng phân cụm.

Do đó, để khắc phục hạn chế trên, một số nghiên cứu cải tiến đã được đề xuất nhằm tự động hóa quá trình lựa chọn d_c . Trong đó, thuật toán CSDPPO (Clustering by Searching Density Peaks based on Parameter Optimization) là một hướng tiếp cận mới, được cải tiến từ CFSFDP, sử dụng hàm Entropy của phân bố mật độ để lựa chọn giá trị d_c phù hợp [9], [14], [15].

2.2.2. Tính mật độ cục bộ bằng Gaussian Kernel

Trong phương pháp truyền thống, mật độ mỗi điểm có thể tính dựa theo công thức Cut-off kernel hoặc Gaussian kernel. Nhưng đối với CSDPPO, thuật toán ưu tiên sử dụng Gaussian kernel để tính toán mật độ cục bộ ρ_i nhằm tận dụng cơ chế cắt mềm, tránh hiện tượng mất mát thông tin biên.

Mật độ cục bộ của mỗi điểm được xác định theo công thức:

$$\rho_i = \sum_{j \neq i} e^{-(d_{ij}/d_c)^2} \quad (2.3)$$

Trong đó:

- d_{ij} là khoảng cách giữa hai điểm dữ liệu i và j ;
- d_c là khoảng cách cắt;
- ρ_i là mật độ cục bộ của điểm i .

2.2.3. Hàm Entropy mật độ

Sau khi thu được phân bố mật độ của toàn bộ tập dữ liệu, hàm Entropy mật độ (H) được thiết lập nhằm đo lường mức độ hỗn loạn hoặc tính đồng đều của dữ liệu [14], [15], cụ thể qua công thức sau:

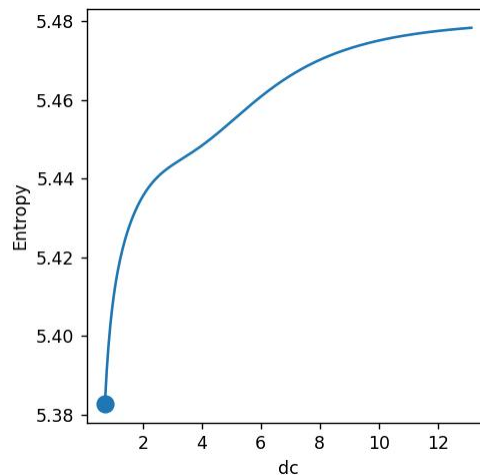
$$H(d_c) = - \sum_{i=1}^N \frac{\rho_i}{Z} \log \left(\frac{\rho_i}{Z} \right) \quad (2.6)$$

Trong đó, N là tổng số điểm dữ liệu của tập nền và Z đóng vai trò là hằng số chuẩn hóa, đại diện cho tổng mật độ cục bộ của tất cả các phần tử trong không gian dữ liệu:

$$Z = \sum_{i=1}^N \rho_i \quad (2.7)$$

Về mặt vận hành, thuật toán CSDPPO sẽ thử nhiều giá trị d_c tiềm năng khác nhau, sau đó tính toán giá trị Entropy tương ứng và lựa chọn giá trị d_c làm Entropy nhỏ nhất để thực hiện tính toán và phân cụm [14]. Ý tưởng của phương pháp này là khi giá trị Entropy nhỏ nhất, sự phân bố mật độ giữa các điểm trở nên rõ ràng hơn, các tâm cụm sẽ có mật độ cao và xuất hiện nổi bật hơn trên Decision Graph, trong khi phần lớn các điểm còn lại có mật độ thấp và phân bố gần nhau hơn để từ đó dễ dàng xác định các tâm cụm hơn trên Decision Graph.

Để minh họa quá trình lựa chọn tham số d_c tối ưu, thuật toán tiến hành khảo sát sự thay đổi của giá trị Entropy theo từng giá trị d_c ứng viên. Kết quả được biểu diễn dưới dạng đồ thị quan hệ giữa Entropy và d_c :



Hình 2.5. Minh họa sự thay đổi của giá trị Entropy theo d_c

Hình 2.5 minh họa sự thay đổi của giá trị Entropy theo tham số khoảng cách cắt d_c . Thông qua đồ thị này, thuật toán có thể xác định giá trị d_c tối ưu tương ứng với vị trí Entropy đạt giá trị nhỏ nhất. Giá trị này phản ánh trạng thái mà sự phân bố mật độ giữa các điểm dữ liệu trở nên rõ ràng hơn, hỗ trợ quá trình xác định tâm cụm hiệu quả hơn.

2.2.4. Quy trình thực hiện thuật toán CSDPPO

Thuật toán CSDPPO được thực hiện thông qua các bước chính sau đây:

Bước 1: Tính ma trận khoảng cách

- Đầu tiên, thuật toán tiến hành tính khoảng cách giữa mọi cặp điểm dữ liệu trong tập dữ liệu đầu vào. Thông thường, khoảng cách Euclidean được sử dụng để đo độ tương đồng giữa các điểm.
- Kết quả của bước này là ma trận khoảng cách $D = [d_{ij}]$, trong đó mỗi phần tử d_{ij} biểu diễn khoảng cách giữa điểm dữ liệu i và j .
- Ma trận khoảng cách sẽ được sử dụng xuyên suốt quá trình tính mật độ cục bộ, xác định Entropy và tính khoảng cách δ .

Bước 2: Tìm giá trị d_c tối ưu dựa trên Entropy

- Sau khi xây dựng ma trận khoảng cách, thuật toán tiến hành tìm giá trị khoảng cách cắt d_c tối ưu.
- Quá trình này được thực hiện như sau:
 - Thuật toán sinh ra nhiều giá trị d_c ứng viên trong một khoảng xác định trước;
 - Với mỗi giá trị d_c , thuật toán sử dụng Gaussian kernel để tính mật độ cục bộ ρ_i cho toàn bộ điểm dữ liệu;
 - Sau đó, giá trị Entropy của phân bố mật độ được tính toán tương ứng với từng d_c .
- Giá trị Entropy phản ánh mức độ phân tán của phân bố mật độ:
 - Entropy càng nhỏ cho thấy sự khác biệt giữa các vùng mật độ càng rõ ràng;
 - Các tâm cụm sẽ xuất hiện nổi bật hơn trên Decision Graph.
- Sau khi hoàn tất quá trình thử nghiệm, thuật toán lựa chọn giá trị d_c tạo ra Entropy nhỏ nhất làm tham số tối ưu cho quá trình phân cụm.

Bước 3: Tính mật độ cục bộ ρ với giá trị d_c tối ưu

- Sau khi xác định được giá trị d_c tối ưu, thuật toán tiến hành tính lại mật độ cục bộ ρ_i cho toàn bộ dữ liệu bằng Gaussian kernel.
- Bước này là cần thiết vì:
 - Trong bước tối ưu Entropy, các giá trị mật độ chỉ mang tính tạm thời nhằm đánh giá từng tham số d_c ;
 - Sau khi xác định được giá trị tối ưu cuối cùng, thuật toán cần tính lại mật độ chính thức để sử dụng cho các bước phân cụm tiếp theo.
- Kết quả của bước này là tập giá trị mật độ cục bộ cuối cùng của toàn bộ dữ liệu.

Bước 4: Tính khoảng cách δ

- Sau khi hoàn tất việc tính mật độ cục bộ, thuật toán tiến hành xác định giá trị δ_i cho từng điểm dữ liệu.
- Đối với mỗi điểm:

- Thuật toán tìm các điểm có mật độ lớn hơn điểm đang xét;
- Sau đó xác định khoảng cách nhỏ nhất từ điểm hiện tại đến các điểm có mật độ cao hơn đó.
- Riêng đối với điểm có mật độ lớn nhất trong toàn bộ tập dữ liệu:
 - Giá trị δ được gán bằng khoảng cách lớn nhất từ điểm đó đến các điểm còn lại.
- Tham số δ phản ánh mức độ tách biệt của điểm dữ liệu trong không gian mật độ.

Bước 5: Xây dựng Decision Graph

- Sau khi tính toán hoàn tất cặp tham số (ρ, δ) , thuật toán biểu diễn toàn bộ dữ liệu trên biểu đồ hai chiều gọi là Decision Graph.
- Trong biểu đồ:
 - Trục hoành biểu diễn mật độ cục bộ ρ ;
 - Trục tung biểu diễn khoảng cách δ .
- Các điểm có:
 - Giá trị ρ cao;
 - Và giá trị δ lớn;

thường xuất hiện nổi bật ở khu vực phía trên bên phải của biểu đồ và được xem là các tâm cụm tiềm năng.

Bước 6: Xác định tâm cụm

- Dựa trên Decision Graph, thuật toán tiến hành lựa chọn các điểm có giá trị ρ và δ lớn làm tâm cụm.
- Quá trình lựa chọn có thể:
 - Thực hiện trực quan thông qua biểu đồ;
 - Hoặc áp dụng thêm các ngưỡng điều kiện để tự động xác định tâm cụm.
- Các điểm được chọn sẽ đóng vai trò đại diện cho từng cụm dữ liệu.

Bước 7: Gán nhãn cụm cho các điểm dữ liệu

- Sau khi xác định được các tâm cụm, thuật toán tiến hành gán nhãn cụm cho các điểm còn lại.
- Mỗi điểm dữ liệu sẽ được gán vào cùng cụm với:
 - Điểm gần nhất có mật độ cao hơn nó.
- Quá trình này tiếp tục cho đến khi tất cả các điểm dữ liệu đều được phân vào một cụm xác định.

Để làm rõ sự khác biệt giữa thuật toán CFSFDP truyền thống và phương pháp cải tiến CSDPPO, Bảng 2.1 trình bày một số đặc điểm nổi bật giữa hai thuật toán theo các tiêu chí về lựa chọn tham số, phương pháp tính mật độ và chi phí tính toán.

Bảng 2.1. So sánh CFSFDP và CSDPPO

Tiêu chí	CFSFDP	CSDPPO
Lựa chọn d_c	Thủ công	Tối ưu bằng Entropy
Hàm mật độ	Cut-off/Gaussian	Gaussian
Tính ổn định	Phụ thuộc tham số	Ổn định hơn
Chi phí tính toán	Cao	Cao
Khả năng tự động hóa	Thấp	Cao hơn

Thông qua Bảng 2.1 có thể thấy, CSDPPO đã cải thiện đáng kể khả năng lựa chọn tham số khoảng cách cắt d_c so với CFSFDP truyền thống. Việc sử dụng hàm Entropy giúp quá trình xác định tham số trở nên tự động và khách quan hơn, góp phần nâng cao tính ổn định của kết quả phân cụm. Tuy nhiên, cả hai thuật toán vẫn tồn tại hạn chế chung về chi phí tính toán lớn trong bước xác định khoảng cách δ .

Mã giả thuật toán CSDPPO được trình bày như sau:

Algorithm 2: CSDPPO Clustering Algorithm

Input:

$D = \{x_1, x_2, \dots, x_n\}$: tập dữ liệu

$DC = \{d_{c1}, d_{c2}, \dots, d_{cn}\}$: tập giá trị d_c ứng viên

Output:

C : tập các cụm dữ liệu

-
- (1) Compute distance matrix D_{ij}
 - (2) for each $d_c \in DC$ do
 - (3) for each point $x_i \in D$ do
 - (4) Compute local density ρ_i using Gaussian kernel
 - (5) end for
 - (6) Compute entropy $H(d_c)$
 - (7) end for
 - (8) Select optimal d_c corresponding to minimum entropy
 - (9) Recompute local density ρ_i using optimal d_c
 - (10) for each point $x_i \in D$ do
 - (11) Find nearest point having higher density
 - (12) Compute δ_i
 - (13) end for
 - (14) Construct Decision Graph using (ρ_i, δ_i)
 - (15) Select cluster centers
 - (16) for each non-center point x_i do
 - (17) Assign x_i to the same cluster as its nearest higher-density neighbor

- (18) end for
- (19) Return clustering result C

2.2.5. Đánh giá thuật toán CSDPPO

Mặc dù bước tối ưu tham số giúp cải thiện chất lượng phân cụm, tuy nhiên quá trình thử nhiều giá trị d_c khác nhau cũng làm gia tăng chi phí tính toán của thuật toán. Với mỗi giá trị d_c , thuật toán cần thực hiện lại quá trình tính mật độ cục bộ cho toàn bộ dữ liệu. Đồng thời, bước xác định khoảng cách δ vẫn yêu cầu so sánh khoảng cách giữa nhiều cặp điểm dữ liệu.

Do đó, độ phức tạp tính toán của thuật toán vẫn ở mức cao:

$$O(N^2)$$

Khi kích thước dữ liệu tăng lớn, thời gian xử lý và số lượng phép tính khoảng cách sẽ tăng đáng kể, ảnh hưởng trực tiếp đến hiệu năng của thuật toán.

Như vậy, thuật toán CSDPPO đã góp phần giúp giảm sự phụ thuộc vào việc lựa chọn thủ công tham số d_c , từ đó cải thiện tính ổn định của kết quả phân cụm trên nhiều bộ dữ liệu khác nhau. Việc sử dụng hàm Entropy cũng giúp quá trình xác định tham số trở nên khách quan và tự động hơn.

Tuy nhiên, dù đã cải thiện ở bước lựa chọn tham số, thuật toán vẫn còn tồn tại một hạn chế chúng ta đã nói trước đó, đó là hạn chế về chi phí tính toán trong bước xác định khoảng cách δ . Khi kích thước dữ liệu tăng lên, số phép tính khoảng cách giữa các điểm vẫn rất cao, làm ảnh hưởng đến thời gian xử lý của thuật toán. Đây cũng là vấn đề được tập trung cải tiến cũng như lý do tôi viết đề tài này.

2.3. ĐỊNH HƯỚNG NGHIÊN CỨU CỦA ĐỀ TÀI

Qua các nội dung đã trình bày, có thể thấy rằng thuật toán CFSFDP [9] có khả năng phát hiện các cụm dữ liệu với hình dạng phức tạp và không cần xác định trước số lượng cụm. Nhưng dù vậy thuật toán vẫn còn tồn tại hai hạn chế liên quan đến việc lựa chọn thủ công tham số khoảng cách cắt d_c và chi phí tính toán trong quá trình xác định khoảng cách δ .

Thuật toán CSDPPO [14] đã cải thiện bước lựa chọn tham số khoảng cách cắt d_c bằng cách sử dụng giá trị entropy của phân bố mật độ để lựa chọn giá trị phù hợp hơn. Nhờ đó giảm sự phụ thuộc vào kinh nghiệm lựa chọn của con người, và có thể cải thiện chất lượng phân cụm trên một số bộ dữ liệu. Tuy nhiên, thuật toán mới chỉ cải thiện được vấn đề lựa chọn tham số khoảng cách cắt, sau bước tìm d_c , thuật toán vẫn phải thực hiện nhiều lần các bước so sánh khoảng cách trong quá trình xác định δ , khi số lượng dữ liệu tăng lên, thời gian xử lý cũng tăng đáng kể.

Xuất phát từ vấn đề trên, đề tài tập trung nghiên cứu cải tiến bước tính khoảng cách δ trong thuật toán gốc CSDPPO nhằm giảm chi phí tính toán nhưng vẫn giữ nguyên kết quả phân cụm của thuật toán gốc.

Trong đề tài này, phương pháp PGD (Pruned Grid Delta) được đề xuất nhằm hỗ trợ tối ưu quá trình tìm kiếm điểm có mật độ lớn hơn gần nhất. Ý tưởng chính của phương pháp này là chia không gian dữ liệu thành các ô lưới (grid) và sử dụng cơ chế loại bỏ (pruning) để giảm số lượng các điểm dữ liệu mà chúng ta cần phải so sánh trực tiếp ban đầu. Thông qua việc giảm số lượng phép so sánh khoảng cách không cần thiết, phương pháp đề xuất hướng tới mục tiêu cải thiện hiệu năng xử lý của thuật toán trên các tập dữ liệu có kích thước lớn, đồng thời vẫn đảm bảo giữ nguyên cơ chế phân cụm và chất lượng kết quả của thuật toán CSDPPO ban đầu.

2.4. ĐỘ ĐO KHOẢNG CÁCH

Trong các bài toán phân cụm dữ liệu, độ đo khoảng cách được sử dụng để xác định mức độ tương đồng hoặc khác biệt giữa các đối tượng dữ liệu [3]. Các điểm có khoảng cách càng nhỏ thì mức độ tương đồng càng cao, ngược lại khoảng cách càng lớn càng thể hiện sự khác biệt lớn hơn giữa các điểm dữ liệu.

Hiện nay có nhiều loại độ đo khoảng cách khác nhau như khoảng cách Euclidean, Manhattan [6],... Tùy thuộc vào đặc điểm dữ liệu và yêu cầu của bài toán mà có thể lựa chọn độ đo phù hợp. Trong các phương pháp phân cụm dựa trên mật độ như CFSFDP [9] và CSDPPO [14], khoảng cách Euclidean thường được sử dụng do có cách tính đơn giản và phù hợp với dữ liệu trong không gian nhiều chiều.

Trên phương diện toán học, khoảng cách Euclidean là độ dài nối giữa hai điểm trong không gian. Trong không gian hai chiều, khoảng cách giữa hai điểm $A(x_1, y_1)$ và $B(x_2, y_2)$ được tính như sau:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (2.8)$$

Trong các thuật toán CFSFDP và CSDPPO, khoảng cách Euclidean được sử dụng trong quá trình tính toán ma trận khoảng cách lúc đầu. Từ đó làm cơ sở cho các bước sau để tính được mật độ cục bộ ρ cũng như xác định khoảng cách δ đến điểm có mật độ lớn hơn gần nhất. Ngoài ra, trong phương pháp PGD được đề xuất trong đề tài này, khoảng cách đến hộp bao (Bounding Box) của từng ô lưới cũng được tính dựa trên khoảng cách Euclidean.

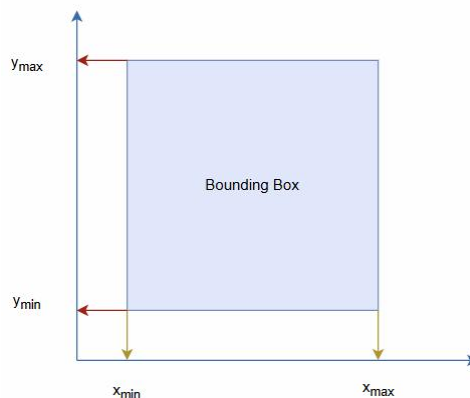
2.5. GRID VÀ BOUNDING BOX

Trong các bài toán xử lý dữ liệu không gian, phương pháp chia lưới (grid) thường được sử dụng nhằm giảm phạm vi tìm kiếm và hỗ trợ tăng tốc các phép xử lý trên dữ liệu nhiều điểm [16]. Ý tưởng chính của phương pháp này là chia không gian dữ liệu thành nhiều ô lưới có kích thước cố định, sau đó lưu trữ các điểm dữ liệu tương ứng vào từng ô.

Khi dữ liệu được tổ chức theo dạng grid, quá trình tìm kiếm có thể được thực hiện trên từng vùng lân cận thay vì phải xét toàn bộ tập dữ liệu. Điều này giúp giảm số lượng đối tượng cần kiểm tra trong nhiều bài toán liên quan đến khoảng cách và truy vấn không gian.

Trong đề tài này, phương pháp chia lưới (grid) được sử dụng nhằm hỗ trợ quá trình tìm kiếm điểm có mật độ lớn hơn gần nhất trong bước tính δ . Không gian dữ liệu được chia thành các ô lưới với kích thước cố định và phụ thuộc vào tham số d_c . Mỗi ô lưới sẽ lưu trữ: danh sách các điểm dữ liệu thuộc ô và thông tin hộp bao (Bounding Box) của ô.

Bounding Box là loại hình chữ nhật bao quanh toàn bộ các điểm dữ liệu hoặc đối tượng trong không gian [16]. Trong không gian dữ liệu nhiều chiều, bounding box của một ô lưới được xác định thông qua các giá trị cực tiểu và cực đại trên từng chiều dữ liệu. Đối với trường hợp dữ liệu hai chiều, Bounding Box có dạng: $[x_{min}, x_{max}, y_{min}, y_{max}]$. Trong đó: x_{min}, x_{max} là giới hạn nhỏ nhất và lớn nhất theo trục x , và y_{min}, y_{max} là giới hạn nhỏ nhất và lớn nhất theo trục y . Mô hình hình học và các vách biên của một hộp bao 2D được phác thảo trong hình 2.2:



Hình 2.2. Minh họa hộp bao (Bounding Box)

Trong đề tài này, Bounding Box được sử dụng trong thuật toán để ước lượng khoảng cách nhỏ nhất từ một điểm dữ liệu đến toàn bộ các ô lưới khác.

Trong quá trình cắt tĩa không gian, thuật toán không tính khoảng cách trực tiếp đến từng điểm dữ liệu bên trong ô lưới. Thay vào đó, thuật toán trước tiên ước lượng khoảng cách nhỏ nhất từ điểm truy vấn đến Bounding Box của ô lưới cần xét.

Nếu khoảng cách tối thiểu này đã lớn hơn ngưỡng tìm kiếm hiện tại, toàn bộ ô lưới sẽ bị loại bỏ mà không cần truy xuất các điểm dữ liệu bên trong. Cơ chế này giúp giảm đáng kể số lượng phép tính khoảng cách cần thực hiện.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

3.1. HẠN CHẾ CỦA BƯỚC XÁC ĐỊNH δ TRONG CSDPPO

3.1.1. Quá trình xác định khoảng cách δ

Trong cấu trúc thuật toán của CSDPPO [14], khi Entropy được sử dụng để tối ưu hóa tham số khoảng cách cắt d_c và sau khi hoàn thành bước tính toán mật độ cục bộ ρ thông qua hàm Gaussian Kernel, hệ thống sẽ tiếp tục tiến hành xác định khoảng cách δ cho từng điểm dữ liệu.

Về mặt bản chất, công thức tính δ được kế thừa từ nghiên cứu CFSFDP của Rodriguez và Laio [9], giá trị δ_i của một điểm dữ liệu i đại diện cho khoảng cách ngắn nhất giữa điểm đó và một điểm j bất kỳ trong tập dữ liệu sao cho $\rho_j > \rho_i$.

Đối với điểm có giá trị mật độ cục bộ lớn nhất, vì không có điểm nào có mật độ cao hơn nó, giá trị δ của nó sẽ được gán bằng khoảng cách lớn nhất giữa nó với các điểm khác trong tập dữ liệu:

Thông thường, quy trình tìm kiếm ứng viên sẽ được triển khai theo hai hướng với hai luồng xử lý khác nhau như sau:

- Duyệt tuyến tính: thuật toán thực hiện hai vòng lặp liên tiếp, tại mỗi điểm, thuật toán kiểm tra mật độ các điểm khác có thỏa mãn điều kiện hay không. Nếu thỏa, giá trị khoảng cách sẽ được đưa vào để so sánh nhằm xác định δ .
- Sắp xếp mảng chỉ số: thuật toán tiến hành sắp xếp các điểm theo mật độ giảm dần, các điểm đứng trước có mật độ cao hơn các điểm đứng sau. Và khi đó, để tính khoảng cách δ cho một điểm nằm ở vị trí thứ k trong mảng, thuật toán chỉ cần thực hiện tìm kiếm đảo ngược lên phía trước (từ vị trí $k-1$ về vị trí 0) để tìm điểm gần nhất.

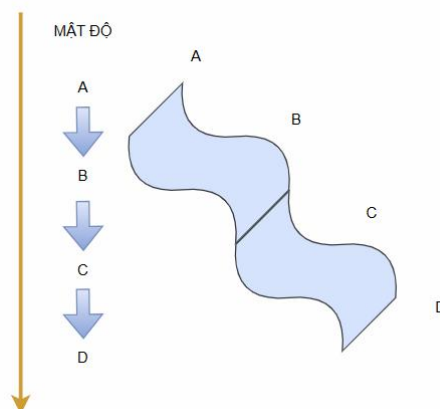
Kết quả của quá trình này là cặp tham số (ρ_i, δ_i) cho từng điểm dữ liệu, phục vụ cho việc xây dựng và biểu diễn các điểm trên đồ thị quyết định (Decision Graph).

3.1.2. Hạn chế về chi phí tính toán

Mặc dù CSDPPO [14] đem lại cải tiến quan trọng trong việc tự động hóa việc tìm tham số khoảng cách cắt d_c thông qua phân bố Entropy, bước xác định khoảng cách δ cốt lõi vẫn giữ nguyên cơ chế duyệt giống với thuật toán gốc CFSFDP [9]. Cơ chế này bộc lộ những hạn chế nghiêm trọng về chi phí vận hành và hiệu năng tính toán khi được triển khai thực tế trên các tập dữ liệu.

Số lượng cặp điểm cần phải duyệt và so sánh điều kiện tăng trưởng theo hàm bậc hai của số lượng phần tử. Ngay cả khi sử dụng phương pháp sắp xếp các điểm theo thứ tự giảm dần mật độ như đã nêu ở 3.1.1, số lượng phép so sánh giá trị khoảng cách để cập nhật biến *best* (biến tìm nghiệm δ) ở trường hợp trung

binh vẫn tiệm cận độ phức tạp $O(N^2)$. Đối với các điểm dữ liệu nằm ở phân vùng có mật độ cục bộ thấp, tức là các điểm nằm cuối danh sách có số lượng ứng viên cần so sánh là cực kỳ lớn. Thuật toán bắt buộc phải thực hiện quét tuyến tính ngược với số lượng phép so sánh điều kiện tăng dần theo cấp số cộng sau mỗi vòng lặp. Sơ đồ Hình 3.1 mô tả quá trình quét tuyến tính ngược của mô hình CSDPPO truyền thống:



Hình 3.1. Minh họa dòng thác mật độ giảm dần

Hình 3.1 cho thấy với các điểm nằm ở cuối danh sách như D, để tìm điểm có mật độ cao hơn gần nhất cần phải đi lên thác để so sánh với các điểm có mật độ cao hơn nó (A, B, C).

Khi kích thước của tập dữ liệu N đạt đến quy mô lớn lên đến hàng chục nghìn, chi phí thời gian dành riêng cho các vòng lặp so sánh lẫn nhau này tăng theo hàm bậc hai. Việc phải duyệt qua cả những vùng không gian dữ liệu quá xa xôi, nơi chắc chắn không thể chứa điểm có nghiệm tốt hơn hiện tại là không cần thiết.

Chính vì vậy, bước xác định δ theo cơ chế tìm kiếm hiện tại của CSDPPO [14] đóng vai trò là tác nhân chính làm giảm hiệu năng tổng thể của hệ thống, làm cho thuật toán không có khả năng đáp ứng tốt các bài toán phân cụm ở quy mô lớn hơn. Đây là lý do, động lực để đề tài này được đề xuất phương pháp phân hoạch và cắt tỉa PGD ở các mục tiếp theo.

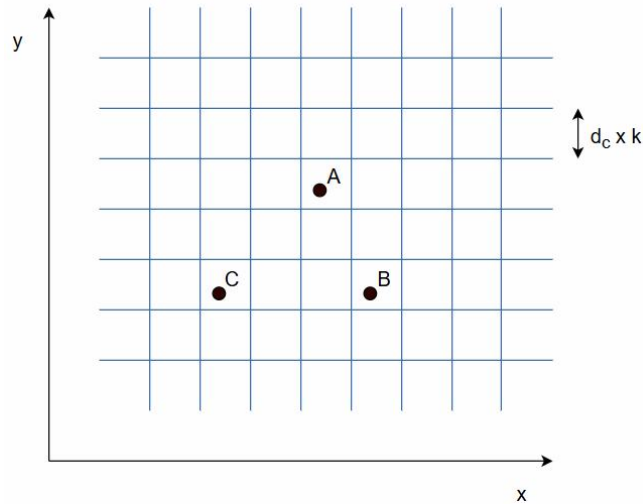
3.2. PHƯƠNG PHÁP PGD ĐỀ XUẤT

3.2.1. Ý tưởng của phương pháp PGD

Để giải quyết hạn chế về mặt thời gian của bước tính δ của thuật toán CSDPPO [14] được nói ở mục 3.1.2, nghiên cứu này đề xuất phương pháp PGD (Pruned Grid Delta). Tư duy cốt lõi của PGD là chuyển đổi từ việc tìm kiếm ứng viên thông qua duyệt tuyến tính sang tìm kiếm có định hướng trong không gian lưới có kết hợp cắt tỉa.

Ý tưởng chủ đạo của phương pháp PGD được xây dựng dựa trên hai kỹ thuật sau:

- Phân hoạch không gian: PGD chia không gian thành các ô lưới đều nhau, mỗi điểm dữ liệu sẽ được gán vào ô lưới nhất định dựa trên tọa độ của điểm đó. Việc này cho phép thuật toán khi tính toán khoảng cách δ cho một điểm dữ liệu, chỉ ưu tiên tìm kiếm các ô thỏa mãn điều kiện cho trước, thay vì phải quét toàn bộ dữ liệu. Cấu trúc số hóa không gian theo hệ số k được trực quan hóa ở Hình 3.2:



Hình 3.2. Minh họa cấu trúc không gian lưới

Nhận xét: trạng thái phân hoạch giúp chuyển đổi bài toán tìm kiếm toàn cục thành bài toán truy vấn vùng lân cận có giới hạn

- Chiến lược cắt tỉa: Các ô lưới hay các vùng không gian không tiềm năng sẽ được loại bỏ nhằm giảm số lượng phép so sánh khoảng cách. PGD thiết lập điều kiện được áp dụng ở quy mô ô lưới. Bộ lọc này dựa trên khoảng cách hình học, nếu khoảng cách gần nhất từ điểm được so sánh đến ô lưới đó lớn hơn giá trị δ tốt nhất hiện tại, ô lưới đó sẽ bị loại bỏ dù cho một số giá trị mật độ lớn nhất trong ô lưới đó lớn hơn mật độ điểm hiện tại. Vì mục tiêu là tìm điểm có mật độ cao hơn gần nhất, các điểm có mật độ lớn hơn nhưng ở quá xa sẽ không được chọn.

3.2.2. Cấu trúc dữ liệu grid

Mỗi ô lưới trong không gian đều có kích thước bằng nhau, kích thước của một ô lưới được xác định dựa trên tham số có sẵn, đó là tham số khoảng cách cắt d_c . Kích thước của ô lưới có thể được thay đổi tùy người sử dụng chỉ định, nhưng công thức chung do phương pháp PGD đề xuất như sau:

$$\text{kích thước ô lưới} = d_c \times k \quad (3.1)$$

Trong đó:

- d_c là khoảng cách cắt được sử dụng trong quá trình tính mật độ;

- k là hệ số điều chỉnh kích thước ô lưới.

Hệ số k giữ vai trò điều chỉnh độ mịn của ô lưới. Nếu k nhỏ, số lượng điểm trong ô lưới sẽ rất thấp, không gian sẽ sinh ra nhiều ô lưới. Trong trường hợp này, mặc dù khả năng cắt tỉa có thể chính xác hơn do phạm vi không gian được chia chi tiết, thuật toán lại phải quản lý số lượng lớn ô lưới và thực hiện nhiều lần truy xuất vùng lân cận. Điều này làm gia tăng chi phí quản lý cấu trúc dữ liệu và ảnh hưởng đến hiệu năng tổng thể.

Còn nếu k lớn, bên trong mỗi ô lưới sẽ chứa rất nhiều điểm dữ liệu. Khi đó, khả năng cắt tỉa của Bounding Box suy giảm do phạm vi bao phủ của mỗi ô quá lớn. Nhiều điểm dữ liệu không liên quan vẫn nằm trong cùng một ô lưới tiềm năng, khiến thuật toán phải thực hiện thêm các phép tính khoảng cách không cần thiết.

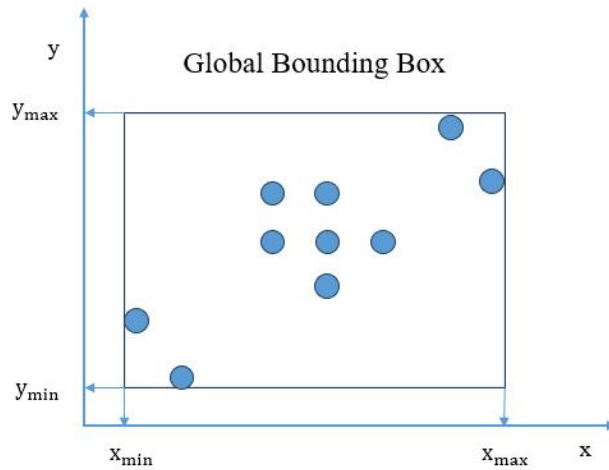
Do đó, hiệu quả của phương pháp PGD phụ thuộc đáng kể vào việc lựa chọn kích thước lưới phù hợp. Một cấu hình lưới hợp lý cần đảm bảo sự cân bằng giữa:

- Số lượng ô lưới được tạo ra;
- Số lượng điểm dữ liệu chứa trong mỗi ô;
- Và khả năng loại bỏ sớm các vùng không gian không tiềm năng

Các ô lưới giữ trọng trách quan trọng trong việc lưu trữ các thông tin nền tảng, ngoài lưu giữ danh sách các điểm bên trong ô, bản thân nó còn chứa thêm giá trị Bounding Box. Trong phương pháp PGD, Bounding Box không được lưu trữ tường minh mà được xác định trực tiếp từ tọa độ và kích thước của ô lưới. Giá trị Bounding Box là giá trị tọa độ của ô trong không gian. Những giá trị này đều góp phần giúp phương pháp PGD hoạt động chính xác, đảm bảo kết quả không bị sai lệch so với thực tế.

3.2.2.1. Global Bounding Box

Để xây dựng hệ thống ô lưới (Grid), bước đầu tiên và đóng vai trò quyết định trong thuật toán là xác định khung giới hạn bao bọc toàn cục (Global bounding box) của tập dữ liệu. Khung giới hạn này được cấu thành từ bốn đường biên ngoài cùng, tạo thành một hình chữ nhật lớn bao trọn toàn bộ các điểm dữ liệu đầu vào. Bằng cách quét qua tọa độ của tất cả các điểm trong không gian khảo sát, thuật toán sẽ xác định được các giá trị cực trị bao gồm: x_{min} , x_{max} , y_{min} , y_{max} như được mô tả trực quan tại Hình 3.3.



Hình 3.3. Sơ đồ minh họa khung giới hạn toàn cục

Ý nghĩa:

- x_{min} và x_{max} : Xác định chiều rộng tối đa của vùng dữ liệu theo trục hoành.
- y_{min} và y_{max} : Xác định chiều cao tối đa của vùng dữ liệu theo trục tung.

3.2.2.2. Tính toán kích thước lưới

Sau khi xác định được khung giới hạn toàn cục (Global bounding box) và cấu trúc của một ô lưới đơn vị (cell size), bước tiếp theo của thuật toán là tính toán số lượng hàng và số lượng cột cần thiết để đảm bảo hệ thống ô lưới phủ kín toàn bộ vùng không gian dữ liệu.

Số lượng cột (đại diện cho chiều ngang) và số lượng hàng (đại diện cho chiều dọc) của lưới được xác định dựa trên tỷ số giữa kích thước tối đa của vùng dữ liệu và kích thước của một ô vuông đơn vị (cell size). Công thức toán học được thiết lập cụ thể như sau:

- Số lượng cột ($Grid_{width}$):

$$Grid_{width} = \left\lceil \frac{x_{max} - x_{min}}{cell_size} \right\rceil + 1$$

- Số lượng hàng ($Grid_{height}$):

$$Grid_{height} = \left\lceil \frac{y_{max} - y_{min}}{cell_size} \right\rceil + 1$$

- Trong đó:

[...] là ký hiệu của phép toán lấy phần nguyên

- Nguyên lý thiết lập công thức:

- Chia lấy nguyên để biết khoảng không gian từ x_{min} đến x_{max} chứa được tối đa bao nhiêu ô nguyên vẹn có kích thước cell_size.

- Cộng thêm 1 để đảm bảo các điểm nằm sát đường biên x_{max} hoặc y_{max} không bị rơi ra ngoài ô lưới. Ô cuối cùng này sẽ hoạt động như một vùng đệm an toàn

Ví dụ minh họa tính biên và kích thước lưới:

- Giả sử có tập dữ liệu gồm 3 điểm sau: $P_1(2, 5)$, $P_2(12, 23)$, $P_3(22, 11)$. Kích thước một ô chọn trước là $cell_size = 5.0$
- Bước 1: Tìm biên của cả lưới:
 - Thuật toán tìm các giá trị nhỏ nhất và lớn nhất:
 - $x_{min} = \min(2, 12, 22) = 2.0$
 - $x_{max} = \max(2, 12, 22) = 22.0$
 - $y_{min} = \min(5, 23, 11) = 5.0$
 - $y_{max} = \max(5, 23, 11) = 23.0$

Hình chữ nhật bao quanh toàn bộ dữ liệu sẽ có góc dưới-trái là (2, 5) và góc trên-phải là (22, 23).

- Bước 2: Tính số hàng và số cột ô lưới
 - Số cột ($grid_width$) = $\left(\frac{22.0-2.0}{5.0}\right) + 1 = 5$ cột
 - Số hàng ($grid_height$) = $\left(\frac{23.0-5.0}{5.0}\right) + 1 = 4$ hàng
 - Các hàng sẽ được đánh chỉ số là: hàng 0, hàng 1, hàng 2, hàng 3

KL: Toàn bộ không gian dữ liệu lúc này được quản lý bởi một ma trận lưới gồm $5 \times 4 = 20$ ô vuông. Bất kỳ điểm dữ liệu nào lọt vào vùng này cũng sẽ tìm được đúng 1 ô cố định để trú ngụ.

3.2.2.3. Cơ chế đánh chỉ số ô lưới

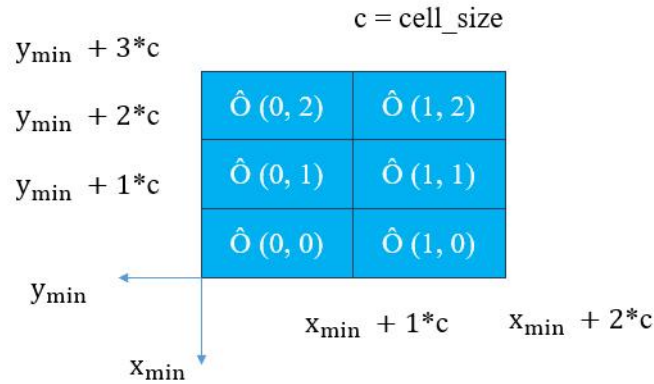
Để quản lý và truy xuất dữ liệu không gian một cách hiệu quả, thuật toán thiết lập một cơ chế đánh chỉ số (indexing) đồng nhất cho toàn bộ hệ thống ô lưới. Hệ thống này sử dụng không gian tọa độ nguyên hai chiều, được định nghĩa bởi cặp chỉ số: (Chỉ số hàng, Chỉ số cột). Góc dưới cùng bên trái của toàn bộ lưới (nơi giao nhau của x_{min} và y_{min}) luôn được quy ước là gốc tọa độ ô: (0, 0).

Nguyên tắc đánh số được quy định cụ thể như sau:

- Gốc tọa độ lưới: Góc dưới cùng bên trái của toàn bộ hệ thống lưới, nơi giao nhau của hai đường biên cực tiểu x_{min} và y_{min} , được quy ước cố định làm gốc tọa độ của các ô lưới với chỉ số là (0, 0).

- Định tiên theo chiều ngang (Trục X): Chỉ số cột tăng dần từ trái sang phải, bắt đầu từ 0 và kết thúc tại $Grid_{width} - 1$.
- Tính tiên theo chiều dọc (Trục Y): Chỉ số hàng tăng dần từ dưới lên trên, bắt đầu từ 0 và kết thúc tại $Grid_{height} - 1$.

Cơ chế định vị và phân bổ chỉ số không gian này được mô tả một cách trực quan trong Hình 3.4.



Hình 3.4. Sơ đồ cơ chế đánh chỉ số ô lưới (Grid Indexing)

3.2.2.4. Xác định một điểm thuộc một ô lưới (Point Mapping)

Sau khi hệ thống lưới và cơ chế đánh chỉ số được thiết lập, bài toán đặt ra là phải xác định chính xác một điểm dữ liệu bất kỳ với tọa độ thực $P(x, y)$ sẽ thuộc về ô lưới (col, row) nào. Quy trình ánh xạ này được thực hiện thông qua hai bước tính toán hình học:

Bước 1: Đo khoảng cách từ gốc lưới

- Thuật toán lấy tọa độ của điểm trừ đi biên tối thiểu toàn cục. Hành động này giúp xác định xem điểm đó đang cách biên trái (hoặc biên dưới) bao nhiêu đơn vị khoảng cách thực tế.

Bước 2: Đếm số lượng ô tròn vẹn

- Lấy khoảng cách vừa tính được chia cho độ dài cạnh của một ô vuông (cell_size).
- Phép chia lấy phần nguyên sẽ cho biết điểm này đã vượt qua được bao nhiêu ô tròn vẹn để đứng ở ô tiếp theo.

Ví dụ trực quan mô phỏng:

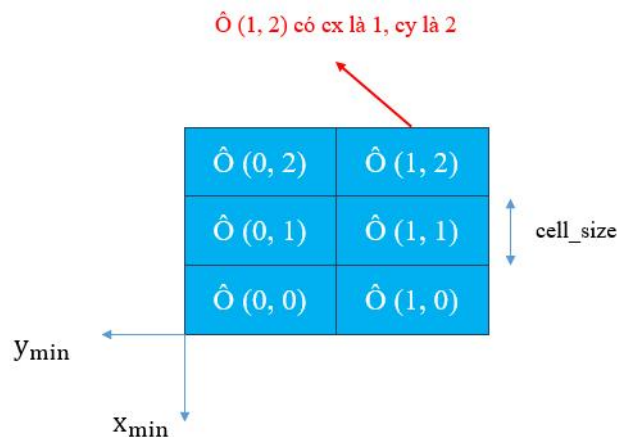
- $x_{\min} = 2.0, y_{\min} = 5.0$
- Kích thước ô: $\text{cell_size} = 5.0$
- Bây giờ hãy tìm vị trí ô cho điểm dữ liệu $P_2(12, 23)$
- 1. Tính chỉ số cột (cell_x)

- Khoảng cách từ P_2 tới biên trái: $12 - 2 = 10$
 - Chia cho kích thước ô: $10 / 5 = 2$
 - $cell_x = 2$ (nằm ở cột số 2)
- 2. Tính chỉ số hàng ($cell_y$)
- Khoảng cách từ P_2 tới biên dưới: $23 - 5 = 18$
 - Chia cho kích thước ô: $18 / 5 = 3.6 \rightarrow$ lấy phần nguyên là 3
 - $cell_y = 3$ (nằm ở hàng số 3)
- KL: Điểm $P_2(12, 23)$ sẽ được xếp vào ô mang số hiệu (2, 3).

3.2.2.5. Xác định biên tọa độ của một ô lưới

Mục tiêu tính toán:

- Khi thuật toán muốn tính khoảng cách nhỏ nhất từ một điểm đến một ô láng giềng ($neighbor_cell$), nó cần phải dựng lên khung hình học (Bounding Box) của ô đó.
- Khung hình học của một ô vuông được định hình bằng 4 đường biên: Biên trái (x_1), Biên phải (x_2), Biên dưới (y_1), Biên trên (y_2). Nguyên lý thiết lập các đường biên này dựa vào chỉ số ô vị trí và kích thước ô vuông ($cell_size$) được mô tả trực quan trong Hình 3.5.



Hình 3.5. Sơ đồ hình học xác định các đường biên tọa độ (x_1, x_2, y_1, y_2) của một ô lưới từ chỉ số (col, row)

Công thức tính: Từ tọa độ lưới nguyên (cx, cy) của ô, thuật toán tính toán 4 biên thực tế như sau:

$$x_1 = x_{min} + cx * cell_size$$

$$y_1 = y_{min} + cy * cell_size$$

$$x_2 = x_1 + cell_size$$

$$y_2 = y_1 + cell_size$$

Trong đó:

- cx là chỉ số cột của ô lưới đó
- cy là chỉ số hàng của ô lưới đó

Ví dụ minh họa:

Giả sử cấu hình lưới toàn cục của chúng ta là:

- Góc lưới: $x_{min} = 10, y_{min} = 20$
- Kích thước ô vuông: $cell_size = 5$

Hãy tính các biên x_1, x_2, y_1, y_2 của ô láng giềng có tọa độ ô lưới là (1, 2) (Cột 1, Hàng 2)

- Bước 1: Xác định vùng không gian theo trục x:
 - Biên trái (x_1) = $10 + (1 \times 5) = 15$
 - Biên trên (x_2) = $15 + 5 = 20$
 - Ô này chiếm giữ vùng không gian nằm ngang từ $[15 - 20]$.
- Bước 2: Xác định vùng không gian theo trục y:
 - Biên trái (y_1) = $20 + (2 \times 5) = 30$
 - Biên trên (y_2) = $30 + 5 = 35$
 - Ô này chiếm giữ vùng không gian thẳng đứng từ $[30 - 35]$.

3.2.3. Pruning theo Bounding Box

Phương pháp PGD áp dụng bộ lọc cắt tia dựa trên khoảng cách hình học không gian, gọi là Pruning theo Bounding Box. Mục tiêu của bộ lọc này là loại bỏ sớm các ô lưới nằm ở các vùng không gian quá xa, chắc chắn không thể chứa nghiệm nào cho giá trị khoảng cách δ ngắn hơn nghiệm tối ưu hiện tại.

Gọi $best$ là giá trị khoảng cách ngắn nhất tính từ điểm x_i đang xét đến một điểm dữ liệu có mật độ cao hơn ($\rho_j > \rho_i$) đã được tìm thấy tính đến thời điểm hiện tại. Tại mỗi bước mở rộng bán kính lưới, trước khi tiến hành duyệt chi tiết các điểm bên trong một ô lưới C , thuật toán sẽ tính toán khoảng cách ngắn nhất từ điểm x_i đang xét đến hộp Bounding Box của ô lưới đó, ký hiệu là $d_{min}(x_i, C)$. Nguyên lý cắt tia theo khoảng cách hình học như sau:

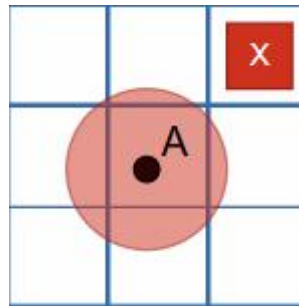
$$d_{min}(x_i, C) > best$$

Nếu điều kiện trên được thỏa mãn, về mặt toán học, khoảng cách từ x_i đến điểm nằm ở vị trí biên gần nhất của ô lưới C đã lớn hơn khoảng cách tới ứng viên tốt nhất hiện tại. Do đó, tất cả các điểm dữ liệu thực tế nằm sâu bên trong ô lưới C chắc chắn sẽ có khoảng cách đến x_i lớn hơn giá trị $best$:

$$\forall x_i \in C, d_{ij} \geq d_{min}(x_i, C) > best$$

Hệ quả là ô lưới C này không còn cơ hội chứa bất kỳ phần tử nào có thể hạ thấp giá trị δ_i hiện tại xuống thêm nữa. Và do vậy, ngay khi điều kiện này được xác định, thuật toán sẽ áp dụng bộ lọc cắt tỉa Bounding Box và cắt bỏ hoàn toàn ô lưới C . Toàn bộ danh sách các điểm bên trong ô lưới bị bỏ qua mà không cần hệ thống phải truy xuất gì nữa.

Cơ chế này đã giúp thuật toán giới hạn vùng tìm kiếm hình học theo một bán kính bằng giá trị $best$ tâm x_i . Khi thuật toán tìm thấy một ứng viên có khoảng cách rất gần ở giai đoạn đầu, giá trị $best$ này giảm xuống nhanh chóng, và kéo theo một lượng lớn các ô lưới ở các tầng bán kính xa hơn bị bộ lọc Bounding Box này chặn đứng và loại bỏ. Điều này giúp tiết kiệm tối đa số lượng phép tính so sánh giá trị khoảng cách trên toàn bộ hệ thống. Chiến lược thiết lập vòng tròn giới hạn và cắt tỉa các ô lưới không tiềm năng được mô tả chi tiết trong hình 3.6:



Hình 3.6. Minh họa vòng tròn cắt tỉa hình học

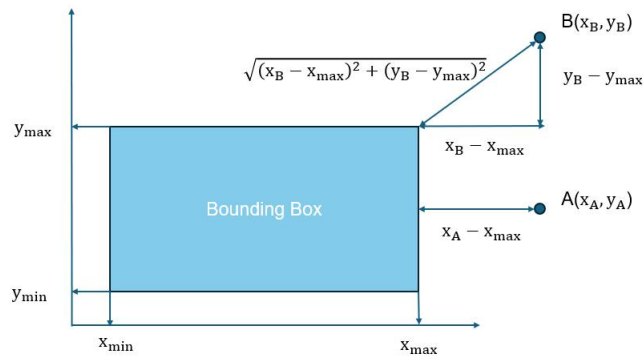
Hình 3.6 chứng minh trực quan rằng toàn bộ các điểm nằm sâu trong ô lưới bị gạch chéo đều không có cơ hội hạ thấp giá trị tối ưu hiện tại.

3.2.3.1. Vai trò của biên ô trong cơ chế cắt tỉa

Ý nghĩa cốt lõi của việc xác định 4 đường biên $[x_1, x_2]$ và $[y_1, y_2]$ của một ô lưới là phục vụ cho cơ chế cắt tỉa (Pruning) trong bài toán tìm kiếm không gian. Thay vì phải tính toán khoảng cách đến từng điểm dữ liệu đơn lẻ bên trong ô, thuật toán sử dụng các đường biên này để xác định khoảng cách ngắn nhất có thể (d_{min}) từ một điểm truy vấn đến rìa của ô lưới đang xét:

- Nếu khoảng cách tới cái rìa này mà còn lớn hơn hoặc bằng khoảng cách tốt nhất hiện tại ($d_{min} \geq best$), thuật toán lập tức đóng cửa ô này lại, không cho phép duyệt các điểm bên trong nữa.
- Phép tính biên này chỉ tốn chi phí $O(1)$ nhưng giúp thuật toán cắt tỉa hàng ngàn phép tính khoảng cách giữa các cặp điểm.

Hình 3.7 minh họa cách xác định khoảng cách ngắn nhất từ một điểm dữ liệu đến Bounding Box trong không gian hai chiều.

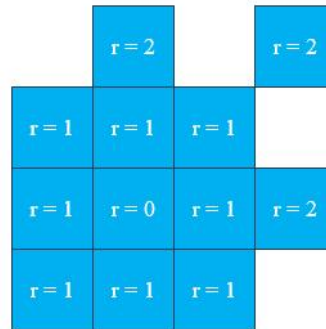


Hình 3.7. Minh họa khoảng cách từ điểm truy vấn đến Bounding Box

3.2.3.2. Chiến lược mở rộng không gian tìm kiếm theo bán kính đồng tâm

Để tối ưu hóa tốc độ xử lý, thuật toán không tiến hành quét qua toàn bộ hệ thống lưới một cách tuần tự. Thay vào đó, không gian tìm kiếm được mở rộng một cách cục bộ từ trong ra ngoài, xuất phát từ ô chứa điểm truy vấn hiện tại và lan rộng theo từng lớp vỏ hình học đồng tâm được định nghĩa bởi bán kính nguyên r (với $r = 0, 1, 2, \dots$).

Cấu trúc phân tầng và quy luật mở rộng của các lớp vỏ không gian này được mô tả trực quan thông qua Hình 3.8.



Hình 3.8. Minh họa cơ chế mở rộng không gian tìm kiếm theo các lớp vỏ bán kính đồng tâm (r)

Không gian tìm kiếm tại mỗi bước lặp được phân cấp nghiêm ngặt dựa trên giá trị của bán kính r :

- Khi $r = 0$: thuật toán chỉ xét chính ô chứa điểm hiện tại.
- Khi $r = 1$: thuật toán quét lớp vỏ sát vách gồm 8 ô xung quanh.
- Khi $r = 2$: thuật toán quét lớp vỏ tiếp theo gồm 16 ô xung quanh.
- Tổng quát: Tại một bán kính r , số lượng ô nằm trên viền vỏ cần quét là $8 \times r$ ô.

3.2.3.3. Thứ tự duyệt chi tiết

Khi thuật toán mở rộng phạm vi tìm kiếm sang lớp vỏ thứ nhất ($r = 1$), một cơ chế duyệt tuần tự được thiết lập để kiểm tra lần lượt 8 ô lưới xung quanh ô

chứa điểm truy vấn hiện tại. Quá trình này được thực hiện dựa trên quy luật quét hình học thông qua các giá trị độ lệch chỉ số (offset) theo hai trục tọa độ lưới.

Thuật toán sử dụng hai biến độ lệch là dx (độ lệch cột) và dy (độ lệch hàng). Quy luật quét hình học: Cố định cột (dx), quét từ dưới lên trên (dy). Xong một cột thì dịch sang phải (tăng dx). Thứ tự này hoàn toàn tuân theo logic toán học của vòng lặp lồng nhau (Nested Loop) trong lập trình cơ bản, chứ chưa có sự ưu tiên theo khoảng cách. Nó cứ bốc lần lượt từng ô theo thứ tự hình học từ trái sang phải như trên để kiểm tra. Ô nào rỗng hoặc quá xa thì bị loại bỏ ngay tại thời điểm gọi tên ô đó. Quét qua 8 ô xung quanh theo chiều mũi tên từ cột trái sang cột phải, trong mỗi cột quét từ dưới lên trên.

Thứ tự chi tiết của 8 ô xung quanh được minh họa trong Bảng 3.1:

Bảng 3.1. Thứ tự duyệt chi tiết của 8 ô xung quanh

Thứ tự xét	Cặp (dx, dy)	Vị trí hình học so với trung tâm
1	(-1, -1)	Phía dưới – bên trái
2	(-1, 0)	Ở giữa – bên trái
3	(-1, 1)	Phía trên – bên trái
4	(0, -1)	Phía dưới – ở giữa
5	(0, 0)	Chính là ô trung tâm (bị thuật toán loại bỏ)
6	(0, 1)	Phía trên – ở giữa
7	(1, -1)	Phía dưới – bên phải
8	(1, 0)	Ở giữa – bên phải
9	(1, 1)	Phía trên – bên phải

Để có cái nhìn chi tiết hơn, thứ tự duyệt có thể được minh họa trong Hình 3.9

Cột trái	Cột giữa	Cột phải
3	6	9
2	5 (ô gốc)	8
1	4	7

Hình 3.9. Minh họa thứ tự duyệt các ô

- **Chú ý:**

- Vị trí ô giống như địa chỉ nhà (ví dụ: Nhà số 5, Đường số 10).

Cặp dx, dy giống như là lời chỉ đường di chuyển (ví dụ: “đi qua trái 1 căn”, “đi qua phải 1 căn”).

3.2.4. Phương pháp PGD

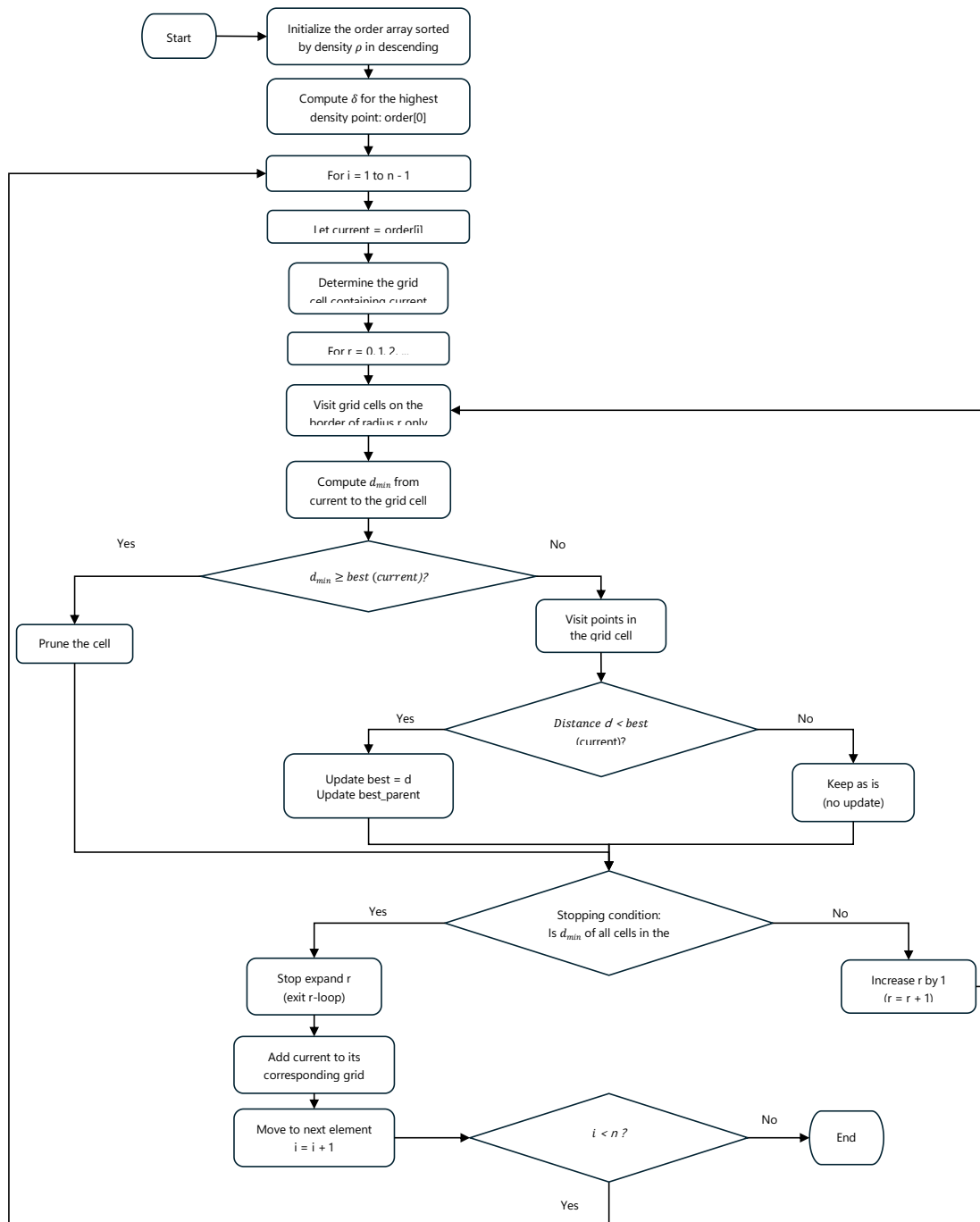
Từ việc kết hợp cấu trúc lưới và chiến lược cắt tia được kết hợp, quy trình tìm kiếm δ cải tiến của phương pháp PGD được mô tả chi tiết thông qua các bước xử lý tuần tự trong sơ đồ khối Hình 3.10:

Bước 1: Khởi tạo cấu trúc lưới

Khi bắt đầu, hệ thống sẽ duyệt qua toàn bộ tập dữ liệu trong không gian để xác định các biên. Dựa trên tham số (*kích thước ô lưới* $= d_c \times k$), thuật toán tiến hành phân hoạch không gian chia không gian dữ liệu thành các ô lưới có kích thước đồng đều. Tại thời điểm khởi đầu, cấu trúc không gian ô lưới ở trạng thái rỗng.

Bước 2: Sắp xếp tập dữ liệu

Toàn bộ các điểm dữ liệu được sắp xếp theo thứ tự giảm dần của giá trị mật độ cục bộ ρ . Trong đó các phần tử đứng trước có mật độ cao hơn sẽ trở thành ứng viên tiềm năng cho các phần tử có mật độ thấp hơn phía sau. Làm cơ sở để so sánh và cập nhật nghiệm cho các điểm có mật độ thấp hơn phía sau.



Hình 3.10. Minh họa lưu đồ của phương pháp PGD

Bước 3: Xác định điểm có mật độ lớn nhất

Thuật toán sẽ tìm điểm có mật độ lớn nhất trên tập dữ liệu. Là phần tử đứng đầu danh sách, điểm có mật độ lớn nhất toàn cục. Vì không có điểm nào có mật độ lớn hơn nữa, giá trị δ của điểm này sẽ được gán bằng khoảng cách lớn nhất giữa điểm này với các điểm khác trong tập dữ liệu: $\delta = \max_{j \neq i} (d_{ij})$. Sau khi xác định được giá trị δ chính thức, điểm gốc này được cập nhật vào danh sách quản lý của các ô lưới tương ứng để chuẩn bị làm ứng viên cho các bước sau.

Bước 4: Vòng lặp tìm kiếm lân cận theo bán kính tăng dần

Với mỗi điểm dữ liệu x_i còn lại, thuật toán khởi tạo giá trị biến $best = \infty$ và xác định chỉ số ô lưới đang chứa điểm x_i (gọi là ô gốc C_{root}). Sau đó thuật toán bắt đầu duyệt các ô lưới xung quanh C_{root} bằng cách mở rộng dần bán kính lưới r ($r = 0, 1, 2, \dots$). Lúc đầu $r = 0$, tức là ô đầu tiên chứa điểm đang xét, thuật toán sẽ so sánh khoảng cách giữa điểm x_i với các điểm có mật độ lớn hơn và tìm giá trị tối ưu nhất sau đó lưu nó vào $best$.

Bước 5: Thực thi bộ lọc cắt tỉa dựa trên Bounding Box

Sau đó thuật toán tiếp tục mở rộng bán kính. Thuật toán tiếp tục lần lượt tính khoảng cách tối thiểu $d_{\min}(x_i, C)$ từ điểm x_i đến Bounding Box của các ô lưới đang xét. Nếu $d_{\min}(x_i, C) > best$, ô lưới C bị loại bỏ hoàn toàn. Toàn bộ danh sách các điểm bên trong ô bị bỏ qua mà không cần hệ thống phải truy xuất hay tính toán chi tiết, giúp tiết kiệm tối đa chi phí vận hành.

Bước 6: Cập nhật nghiệm

Nếu ô lưới C vượt qua điều kiện trên, thuật toán mới bắt đầu truy xuất vào danh sách các điểm dữ liệu nằm trong ô lưới đó. Thuật toán sẽ tính khoảng cách từ điểm hiện tại đến các điểm ứng viên đã được xử lý trước đó trong thứ tự mật độ giảm dần để cập nhật nghiệm, $best = \min(best, d_{ij})$.

Bước 7: Điều kiện để dừng vòng lặp

Vòng lặp mở rộng bán kính r cho điểm x_i dừng lại khi khoảng cách gần nhất từ x_i của ô lưới gốc đến các ô lưới thuộc bán kính tiếp theo vượt quá giá trị $best$ hiện tại. Và khi đó, giá trị δ_i được gán chính thức bằng $best$.

Bước 8: Cập nhật kết quả và chèn điểm vào lưới

Sau khi vòng lặp tìm kiếm dừng lại, giá trị khoảng cách δ_i chính thức của điểm dữ liệu x_i được gán bằng giá trị $best$ thu được. Cuối cùng, phần tử x_i được cập nhật và thêm vào danh sách dữ liệu của ô lưới chứa nó. Thao tác này giúp điểm x_i sẵn sàng đóng vai trò làm ứng viên mật độ cao cho các điểm dữ liệu có mật độ thấp hơn sẽ được xử lý ở các vòng lặp phía sau của hệ thống.

Mã giả của phương pháp PGD được trình bày như sau:

Dựa trên quy trình logic 8 bước đã được mô tả, phương pháp PGD được thể hiện thông qua cấu trúc mã giả dưới đây:

Input:

data, dist_matrix, rho, dc, k

Output:

delta, nearest_higher

```

(1)  Initialize delta and nearest_higher
(2)  cell_size  $\leftarrow$  dc  $\times$  k
(3)  Sort points by descending density  $\rightarrow$  order
(4)  Build empty spatial grid
(5)  Insert highest-density point into grid
(6)  Set its delta as the maximum distance
(7)  for each point p in order do
(8)      best  $\leftarrow +\infty$ 
(9)      r  $\leftarrow$  0
(10)     while r  $\leq$  max_radius do
(11)         Search neighboring cells on boundary r
(12)         for each candidate q in valid cells do
(13)             if MinDistanceToCell  $\geq$  best then
(14)                 continue
(15)             end if
(16)             Compute distance d(p, q)
(17)             if d < best then
(18)                 best  $\leftarrow$  d
(19)                 nearest_higher[p]  $\leftarrow$  q
(20)             end if
(21)         end for
(22)         if no closer candidate can exist then
(23)             break
(24)         end if
(25)         r  $\leftarrow$  r + 1
(26)     end while
(27)     delta[p]  $\leftarrow$  best
(28)     Insert p into grid
(29) end for
(30) return delta, nearest_higher

```

3.2.5. Ví dụ minh họa quá trình xác định δ bằng phương pháp PGD

Để minh họa trực quan cơ chế hoạt động của phương pháp PGD, giả sử ta có tập dữ liệu gồm bốn điểm dữ liệu như trong Bảng 3.2.

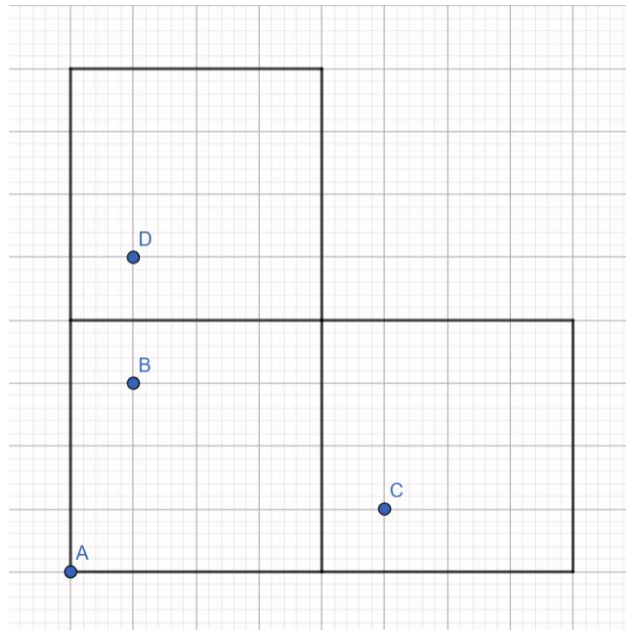
Bảng 3.2. Dữ liệu minh họa cho phương pháp PGD

Điểm	Tọa độ	Mật độ
A	(1, 1)	10
B	(2, 4)	8

C	(7, 2)	5
D	(2, 6)	2

Giả sử kích thước ô lưới: cell_size = 4

Ta có không gian ô lưới như sau, Hình 3.11



Hình 3.11. Minh họa các điểm trên ô lưới

- Biên của cả lưới như sau:

- $x_{min} = \min(1, 2, 7, 2) = 1$
- $x_{max} = \max(1, 2, 7, 2) = 7$
- $y_{min} = \min(1, 4, 2, 6) = 1$
- $y_{max} = \max(1, 4, 2, 6) = 6$
- Hình chữ nhật bao quanh toàn bộ dữ liệu sẽ có góc dưới-trái là (1, 1) và góc trên-phải là (7, 6)

- Tính số hàng và số cột:

- Số cột (grid_width) = $\left(\frac{7-1}{4}\right) + 1 = 2$ cột
- Số hàng (grid_height) = $\left(\frac{6-1}{4}\right) + 1 = 2$ hàng

- Ma trận lưới gồm: $2 \times 2 = 4$ ô vuông

- Chỉ số ô lưới: Dựa theo mục 3.2.2.3, chỉ số ô lưới được xác định như sau: ô lưới chứa điểm A và B là ô (0, 0), ô lưới chứa điểm D là ô (0, 1) và ô lưới chứa điểm C là ô (1, 0).

- Điểm A(1, 1) có:

$$grid_x = \frac{1 - 1}{4} = 0$$

$$grid_y = \frac{1 - 1}{4} = 0$$

nên nằm trong ô (0, 0).

- Điểm B(2, 4) có:

$$grid_x = \frac{2 - 1}{4} = 0$$

$$grid_y = \frac{4 - 1}{4} = 0$$

nên nằm trong ô (0, 0).

- Điểm C(7, 2) có:

$$grid_x = \frac{7 - 1}{4} = 1$$

$$grid_y = \frac{2 - 1}{4} = 0$$

nên nằm trong ô (1, 0)

- Điểm D(2, 6) có:

$$grid_x = \frac{2 - 1}{4} = 0$$

$$grid_y = \frac{6 - 1}{4} = 1$$

nên nằm trong ô (0, 1).

- Sắp xếp các điểm theo mật độ giảm dần: $A \rightarrow B \rightarrow C \rightarrow D$

- Vì điểm A có mật độ cao nhất nên không tồn tại điểm nào có mật độ cao hơn. Do đó:

$$\delta_A = \max(d(A, B), d(A, C), d(A, D)) = d(A, C) \approx 6.08$$

- Sau khi tính toán xong, điểm A được thêm vào ô lưới (0,0).

- Điểm B nằm trong ô (0,0) chứa A là điểm có mật độ cao hơn nó.

- Khoảng cách:

$$d(B, A) \approx 3.16$$

- Khi đó $best = 3.16$

- Thuật toán tiếp tục xét các ô khác $\rightarrow$ tính khoảng cách tới Bounding Box. Thuật toán sẽ tính khoảng cách lần lượt tới 8 ô xung quanh. Các ô thỏa điều kiện:

$$d_{min}(B, \text{ô lưới}) < best$$

sẽ được truy cập.

- Vì các ô xung quanh đều rỗng (các ô rỗng sẽ bị loại bỏ trước mà chưa cần phải xét tới khoảng cách d_{min} đến ô đó, hiện tại D và C chưa được đưa vào ô lưới $\rightarrow$ các ô khác chứa D, C hiện tại đang rỗng) $\rightarrow \delta_B = 3.16 \rightarrow B$ được đưa vào ô (0,0).

- Thuật toán tiếp tục tính δ_C . C nằm trong ô (1,0), không có điểm nào khác ngoài chính nó $\rightarrow$ mở rộng bán kính.

- C truy cập vào ô (0,0) (có thể xem lại thứ tự duyệt ở mục 3.2.3.3, hiện tại chỉ có ô chứa A và B là có tồn tại điểm, nên C truy cập vào ô chứa 2 điểm này):

$$d(C, A) \approx 6.08 \rightarrow best = 6.08$$

$$d(C, B) \approx 5.38, \text{ vì } 5.38 < best \rightarrow best = 5.38$$

- Tương tự, thuật toán tiếp tục xét các ô khác $\rightarrow$ tính khoảng cách tới Bounding Box. Thuật toán sẽ tính khoảng cách lần lượt tới 16 ô xung quanh ($r = 2$).

- Vì các ô xung quanh đều rỗng $\rightarrow \delta_C = 5.38 \rightarrow C$ được đưa vào ô (1,0)

- Thuật toán tiếp tục tính δ_D . D nằm trong ô (0,1), không có điểm nào khác ngoài chính nó $\rightarrow$ mở rộng bán kính.

- Theo thứ tự duyệt thuật toán sẽ truy cập ô chứa A, B trước, sau đó mới đến ô chứa C.

- Thuật toán truy cập vào ô (0,0):

$$d(D, A) \approx 5.09 \rightarrow best = 5.09$$

$$d(D, B) = 2, \text{ vì } 2 < best \rightarrow best = 2$$

- Vì ngoài ô (0,0) chứa A và B, còn một khác là (1,0) chứa điểm C. Trước khi quyết định nên truy cập ô này hoặc loại bỏ, thuật toán sẽ xét d_{min} tới ô (1,0):

$$d_{min}(D, \text{ô lưới chứa } C(1,0)) = \sqrt{(5-2)^2 + (6-5)^2} = \sqrt{10} \approx 3.16$$

Vì $best = 2 \leq d_{min}(D, \text{ô lưới chứa } C(1,0)) = 3.16 \rightarrow \hat{O}(1,0)$ bị cắt tỉa mà không cần phải truy cập vào ô này.

- Thuật toán tiếp tục mở rộng bán kính ($r = 2$), vì các ô đều rỗng, giá trị *best* là nghiệm tốt nhất hiện tại:

$$\delta_D = best = 2$$

3.2.6. Phân tích độ phức tạp

Để chứng minh tính hiệu quả của phương pháp đề xuất PGD so với phương pháp cũ trong việc tìm δ_i của thuật toán CSDPPO, mục này tiến hành bóc tách và phân tích độ phức tạp của phương pháp trên cả hai phương diện: không gian lưu trữ và thời gian xử lý. Việc tích hợp thêm cấu trúc lưới dữ liệu đòi hỏi hệ thống phải cấp phát thêm tài nguyên bộ nhớ để hỗ trợ.

- Độ phức tạp không gian (Space Complexity) có thể tăng lên dựa vào các yếu tố sau: thứ nhất là số lượng ô lưới, số lượng ô lưới phụ thuộc vào cách mà dữ liệu phân bố, các ô không có điểm dữ liệu nào thường bị loại bỏ và không tham gia vào quá trình tính toán. Nhưng trong trường hợp xấu nhất, dữ liệu phân bố đều, và do đó số lượng các ô tăng lên. Ngoài ra, kích thước ô lưới cũng ảnh hưởng đến điều này, nếu kích thước quá bé sẽ sinh ra số lượng ô lưới lớn, nhưng thường không lớn hơn tổng số điểm N . Do cấu trúc này tăng trưởng tuyến tính theo quy mô của tập dữ liệu hoặc số chiều, tổng độ phức tạp không gian đạt mức $O(N \cdot n)$. Về mặt kiến trúc hệ thống, độ phức tạp này hoàn toàn tương thích và không làm tăng bậc độ phức tạp tổng thể, vì vốn thuật toán CSDPPO đã tính toán ma trận khoảng cách có độ phức tạp $O(N^2)$. Và do vậy chi phí bộ nhớ đánh đổi cho PGD là cực kỳ nhỏ và hoàn toàn khả thi.
- Độ phức tạp thời gian (Time Complexity) trong phương pháp PGD có thể được tính qua hai giai đoạn. Vào giai đoạn đầu, thuật toán khởi tạo lưới thực hiện một vòng lặp tuyến tính đi qua N điểm dữ liệu và tiến hành xác định vị trí ô lưới, các điểm trong ô lưới. Tổng chi phí thời gian cho giai đoạn này là $O(N \cdot n)$. Giai đoạn hai là giai đoạn quan trọng nhất, là giai đoạn thực hiện vòng lặp so sánh kết hợp cắt tỉa. Trong trường hợp lý tưởng, số chiều dữ liệu thấp ($n \leq 3$), nhờ chiến lược cắt tỉa, thuật toán chỉ cần duyệt qua ô đầu tiên và các ô lân cận với ($r < 1$). Số lượng ứng viên giảm từ N xuống còn một hằng số c nhỏ. Khi đó, chi phí tính δ_i cho một điểm đạt mức tiệm cận $O(1)$, và tổng thời gian tính toán cho toàn tập dữ liệu giảm xuống mức lý tưởng $O(N)$. Nhưng trong trường hợp xấu nhất, số chiều trong không gian lớn, số lượng ô xung quanh ở mỗi bán kính r sẽ tăng theo hàm bậc mũ $O(3^n - 1)$, các bộ lọc cắt tỉa không còn hoạt động tốt nữa. Và do vậy buộc phải quét qua gần như toàn bộ các ô lưới. Lúc này độ phức tạp trở lại mức như trước $O(N^2)$.

Phương pháp PGD mang lại hiệu năng tốt trên các tập dữ liệu lớn nhưng với số chiều trong không gian vừa phải, thuộc các bài toán không gian địa lý, xử

lý ảnh cơ bản,... Ở các dạng dữ liệu này, PGD hạ thấp số lượng phép so sánh đáng kể giúp thuật toán giảm được lượng thời gian một cách hiệu quả.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. MÔI TRƯỜNG THỰC NGHIỆM

Nhằm đảm bảo tính khách quan, độ chính xác của các kết quả đo lường hiệu năng, toàn bộ các kịch bản thực nghiệm trong nghiên cứu này đều được triển khai trên một hệ thống phần cứng và môi trường phần mềm cố định.

Cấu hình phần cứng là một laptop cá nhân với cấu hình chi tiết như sau:

- Bộ vi xử lý (CPU): Intel Core I5
- Bộ nhớ trong (RAM): 16 GB
- Ổ cứng lưu trữ: SSD

Môi trường phần mềm cụ thể như sau:

- Hệ điều hành: Microsoft Windows 11
- Ngôn ngữ lập trình: Python
- Môi trường phát triển tích hợp (IDE): Visual Studio Code

Các thư viện hỗ trợ: Numpy, Scipy, Scikit-learn, Matplotlib [17].

4.2. DỮ LIỆU THỰC NGHIỆM

Để đánh giá toàn diện hiệu năng của phương pháp PGD, thực nghiệm được tiến hành dựa trên nhiều loại dữ liệu khác nhau. Hai nhóm dữ liệu chuẩn và phổ biến trong khai phá dữ liệu bao gồm tập dữ liệu mô phỏng và tập dữ liệu thế giới thực.

Các tập dữ liệu mô phỏng thường được sử dụng để kiểm tra khả năng nhận diện các cấu trúc cụm có hình dạng phức tạp, phân bố không đồng đều, một trong những bộ dữ liệu mô phỏng có thể được kể đến như sau:

- Aggregation: Bao gồm 7 cụm dữ liệu với các hình dạng và mật độ khác nhau. Một số cụm nằm rất gần nhau, gây khó khăn cho quá trình phân tách cụm
- Flame: Mô phỏng hình dáng đám lửa với 2 cụm dữ liệu chính cùng các điểm nhiễu phân bố xung quanh.
- Spiral: Tập dữ liệu gồm 3 cụm có cấu trúc xoắn ốc lồng vào nhau. Đây là tập dữ liệu kinh điển dùng để thử thách các thuật toán phân cụm.
- Các tập dữ liệu tên tỉnh thành Việt Nam (CanTho, DongNai) và các nhóm (Nhom1, Nhom2, Nhom3): Điểm chung của các tập dữ liệu này là số lượng điểm dữ liệu rất nhiều (N lớn) và khó nhận dạng các cụm chính xác bằng mắt thường.

Các tập dữ liệu thế giới thực thường có số lượng mẫu N lớn và một số bộ dữ liệu còn kèm theo nhiều chiều dữ liệu. Các bộ dữ liệu này được sinh ra nhằm

để đo lường hiệu năng và năng lực giảm thiểu các điểm làm chậm thuật toán. Có hai bộ dữ liệu chính mà đề tài này quan tâm sử dụng bao gồm:

- Iris: Đây là tập dữ liệu phân loại hoa diên vĩ gồm 150 mẫu, 4 thuộc tính. Tập dữ liệu này có số chiều nhỏ, đóng vai trò làm thử nghiệm để xác nhận phiên bản cải tiến giữ nguyên kết quả phân cụm lý thuyết so với CSDPPO[14].

Các bộ dữ liệu trên có thể được liệt kê trong Bảng 4.1 sau:

Bảng 4.1. Tóm tắt các bộ dữ liệu

Loại dữ liệu	Bộ dữ liệu	Số mẫu	Số chiều	Số cụm
Mô phỏng	Aggregation	788	2	7
	Flame	240	2	2
	Spiral	312	2	3
	CanTho	2562	2	3
	DongNai	2703	2	2
	Nhom1	4445	2	5
	Nhom2	4315	2	2
	Nhom3	5648	2	3
Thế giới thực	Iris	150	4	3

4.3. TIÊU CHÍ ĐÁNH GIÁ

4.3.1. Thời gian xử lý

Thời gian xử lý là tiêu chí thực nghiệm trực quan và quan trọng trong việc đánh giá hiệu năng vận hành của thuật toán cải tiến so với thuật toán gốc. Trong phạm vi nghiên cứu của đề án này, tiêu chí này được định nghĩa là khoảng thời gian hệ thống thực thi riêng biệt bước xác định khoảng cách δ cho toàn bộ các điểm trong tập dữ liệu.

Về mặt kỹ thuật, thời gian xử lý được đo lường bằng cách sử dụng thư viện (time) trong ngôn ngữ lập trình Python để ghi lại thời điểm bắt đầu (T_{start}) và thời điểm kết thúc (T_{end}) ngay trước và sau khi khối lệnh tính δ hoạt động. Công thức xác định thời gian thực thi tổng thể (T_{exec}) được tính như sau:

$$(T_{exec})=(T_{end})-(T_{start}) \quad (4.1)$$

Trong đó đơn vị đo lường được sử dụng là giây (s).

Để loại bỏ các yếu tố gây nhiễu ngẫu nhiên từ hệ điều hành như tiến trình chạy ngầm hay trạng thái cấp phát tài nguyên của CPU/RAM tại thời điểm đó,... Mỗi thuật toán được thực hiện nhiều lần trên cùng một bộ dữ liệu và kết quả cuối cùng được lấy trung bình nhằm hạn chế ảnh hưởng của các yếu tố đó.

4.3.2. Số lượng phép so sánh khoảng cách

Trong khi thời gian xử lý có thể bị thay đổi tùy thuộc vào cấu hình phần cứng máy tính và trạng thái phân phối tài nguyên của hệ điều hành tại thời điểm chạy. Số lượng phép so sánh khoảng cách là tiêu chí đánh giá mang tính khách quan và độc lập với thiết bị.

Tiêu chí này phản ánh trực tiếp cấu trúc và bản chất toán học của thuật toán, dùng để chứng minh hiệu quả của phương pháp PGD.

Trong thuật toán gốc CSDPPO, để tìm ra δ_i cho một điểm i , hệ thống buộc phải thực hiện duyệt tuyến tính để kiểm tra điều kiện mật độ với các điểm khác. Số lượng phép so sánh khoảng cách cần thực hiện trong trường hợp xấu nhất có thể ở mức độ phức tạp:

$$\frac{N(N-1)}{2} \approx O(N^2) \quad (4.2)$$

Trong phương pháp cải tiến PGD, hệ thống phân hoạch dữ liệu thành cấu trúc lưới kết hợp với chiến lược cắt tia nhằm giảm số lượng phép so sánh thực tế.

Mục tiêu của tiêu chí này là chứng minh phương pháp PGD có thể ngăn chặn hiệu quả việc truy xuất các vùng không gian xa xôi hoặc không tiềm năng, Bằng cách so sánh tổng số phép toán học thực tế của PGD so với mức $O(N^2)$ của CSDPPO gốc.

Công thức thực hiện như sau:

$$Tỉ lệ cắt tia = 1 - \left(\frac{SSKC_{PGD}}{SSKC_{CSDPPO}} \right) \times 100\% \quad (4.3)$$

Trong đó:

- $SSKC_{PGD}$ là số lượng phép so sánh khoảng cách của phương pháp PGD-CSDPPO;
- $SSKC_{CSDPPO}$ là số lượng phép so sánh khoảng cách của thuật toán CSDPPO gốc.

Tỉ lệ này càng cao càng thể hiện năng lực tối ưu hóa không gian của thuật toán cải tiến, đặc biệt là trên các tập dữ liệu có quy mô lớn.

4.3.3. Độ chính xác phân cụm

Bên cạnh mục tiêu tối ưu hóa về mặt hiệu năng xử lý, một yêu cầu bắt buộc đối với mọi thuật toán cải tiến là phải bảo toàn tính chính xác của kết quả phân

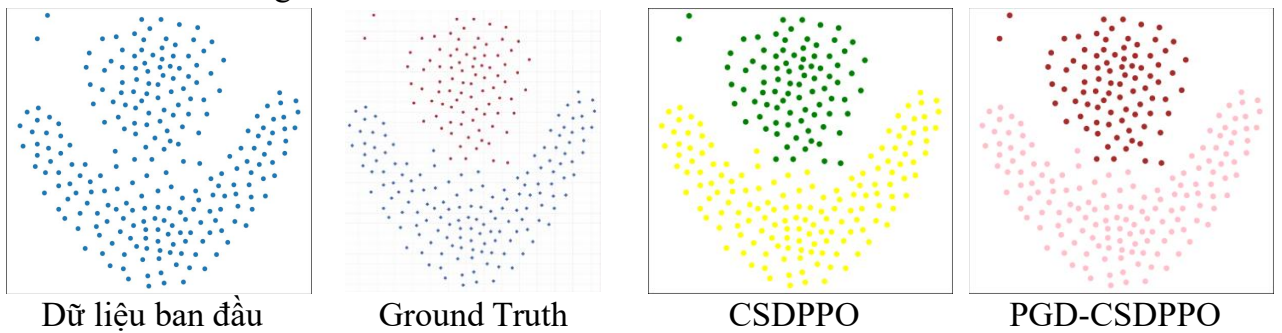
cụm. Phương pháp PGD đề xuất sử dụng các quy tắc cắt tia mang tính hình học chặt chẽ, do đó về mặt lý thuyết, nó không được phép làm mất mát nghiệm tối ưu hoặc làm thay đổi nhãn cụm của bất kỳ điểm dữ liệu nào so với thuật toán gốc CSDPPO.

Để kiểm chứng một cách khách quan, đồ án sử dụng hai chỉ số đánh giá hiệu năng phân cụm chuẩn hóa quốc tế là chỉ số ARI (Adjusted Rand Index) và chỉ số NMI (Normalized Mutual Information). Cả hai chỉ số này đều được tính toán bằng cách đối sánh kết quả phân cụm của thuật toán với nhãn thực tế (Ground Truth) có sẵn của bộ dữ liệu.

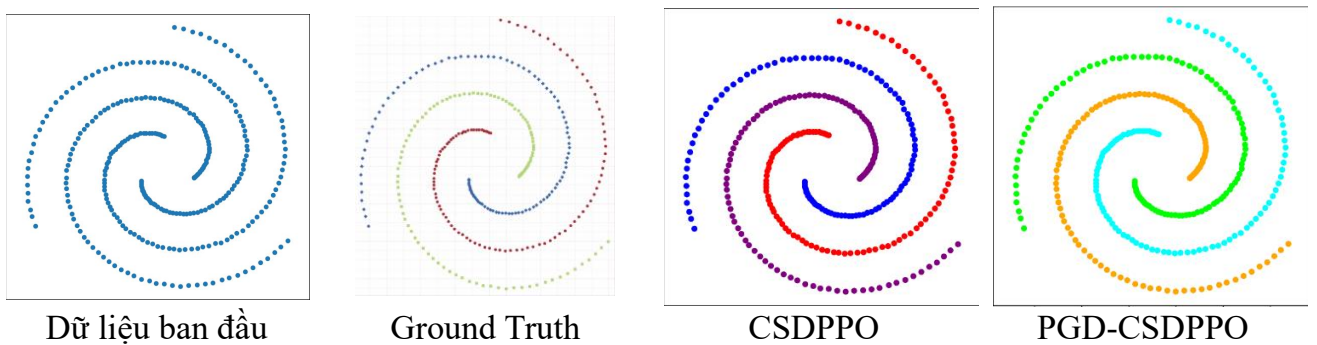
Mục tiêu chính của tiêu chí này là đối chiếu trực tiếp cặp chỉ số (ARI, NMI) của phương pháp PGD cải tiến với thuật toán CSDPPO gốc trên cùng một bộ dữ liệu và cùng một tham số khoảng cách cắt d_c . Nếu các giá trị này trùng khớp hoàn toàn với sai số bằng 0, nghiên cứu sẽ có đầy đủ cơ sở khoa học để khẳng định chiến lược cắt tia kép của PGD là hoàn toàn an toàn và chính xác.

4.4. KẾT QUẢ THỰC NGHIỆM

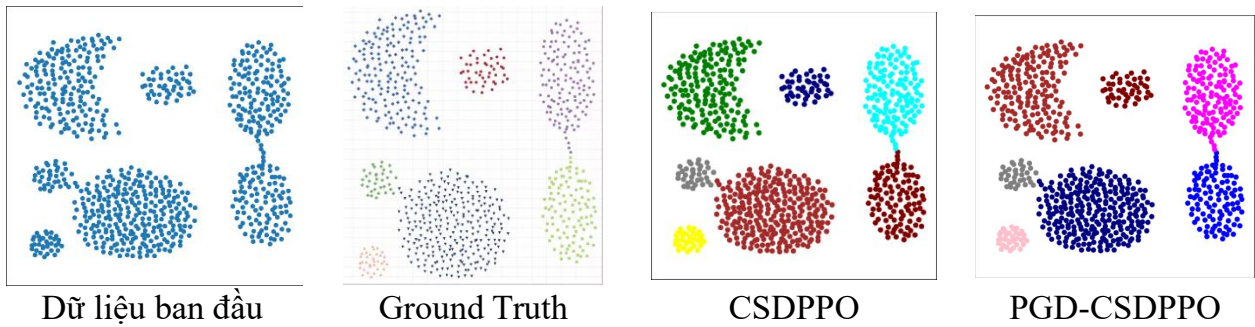
Các kết quả thực nghiệm theo mỗi bộ dữ liệu được thực hiện chi tiết như sau: ảnh đầu biểu thị dữ liệu ban đầu, ảnh thứ hai là dữ liệu nhãn thật ground truth [18], các ảnh tiếp theo tương ứng là ảnh kết quả phân cụm giữa hai thuật toán CSDPPO gốc và PGD-CSDPPO:



Hình 4.1. Kết quả phân cụm bộ dữ liệu Flame



Hình 4.2. Kết quả phân cụm bộ dữ liệu Spiral



Hình 4.3. Kết quả phân cụm bộ dữ liệu Aggregation

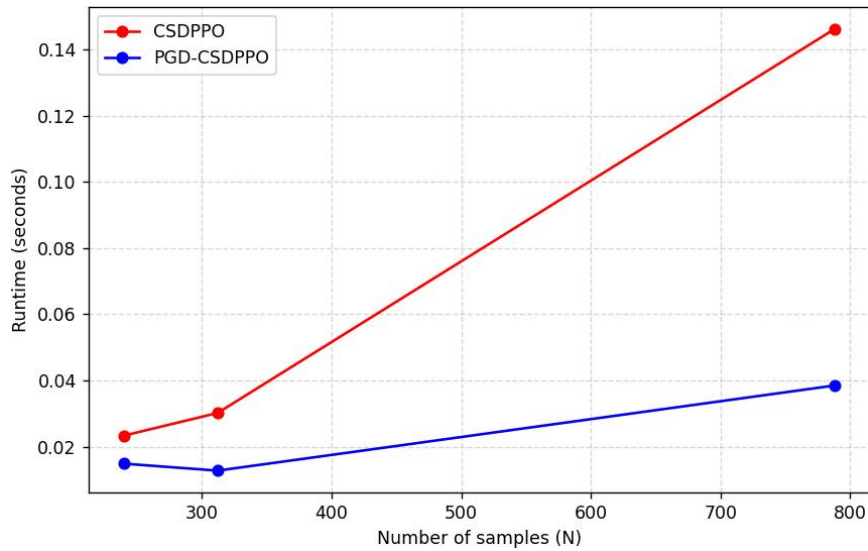
Kết quả thực nghiệm của các bộ dữ liệu thành phố (CanTho, DongNai) và các bộ dữ liệu nhóm (Nhom1, Nhom2, Nhom3) được thể hiện trong bảng 4.2:

Bảng 4.2. Kết quả thực nghiệm các bộ dữ liệu thành phố và dữ liệu nhóm

Datasets	Thuật toán	Silhouette	Runtime (seconds)	% Silhouette vs Original	% Runtime vs Original	No. Clusters
CanTho	Gốc	0.8211	1.8944			3
	Cải tiến	0.8211	0.0765	100%	4%	3
DongNai	Gốc	0.6456	0.9925			2
	Cải tiến	0.6456	0.0673	100%	7%	2
Nhom1	Gốc	0.7040	7.1503			5
	Cải tiến	0.7040	2.4182	100%	34%	5
Nhom2	Gốc	0.6631	2.4563			2
	Cải tiến	0.6631	0.2112	100%	9%	2
Nhom3	Gốc	0.5970	4.3370			3
	Cải tiến	0.5970	0.2246	100%	5%	3

4.4.1. So sánh thời gian xử lý

Dựa trên các kịch bản thực nghiệm đã được nói đến tại mục 4.1 và 4.2, thời gian thực thi bước tính khoảng cách δ của thuật toán CSDPPO gốc và phương pháp cải tiến PGD-CSDPPO được đo đạc trực tiếp. Kết quả thời gian chạy trung bình sau nhiều lần lập trên mỗi bộ dữ liệu được tổng hợp chi tiết trong Bảng 4.3. Để làm rõ xu hướng tốc độ khi quy mô dữ liệu mở rộng, biểu đồ đường so sánh được biểu diễn trong Hình 4.4



Hình 4.4. Đồ thị so sánh thời gian chạy – Runtime

Đường đồ thị Hình 4.4 minh chứng tốc độ xử lý của PGD-CSDPP khá tốt so với giải thuật gốc.

Bảng 4.3: Bảng so sánh thời gian thực thi bước tính δ giữa hai thuật toán

Datasets	Thuật toán	Runtime (seconds)	% Runtime vs Original	Số lượng mẫu (N)	Số chiều	No. Clusters
Flame	Gốc	0.0234		240	2	2
	Cải tiến	0.0149	64%			2
Spiral	Gốc	0.0302		312	2	3
	Cải tiến	0.0128	42%			3
Aggregation	Gốc	0.1461		788	2	7
	Cải tiến	0.0385	26%			7
Iris	Gốc	0.0048		150	4	3
	Cải tiến	0.0042	88%			3

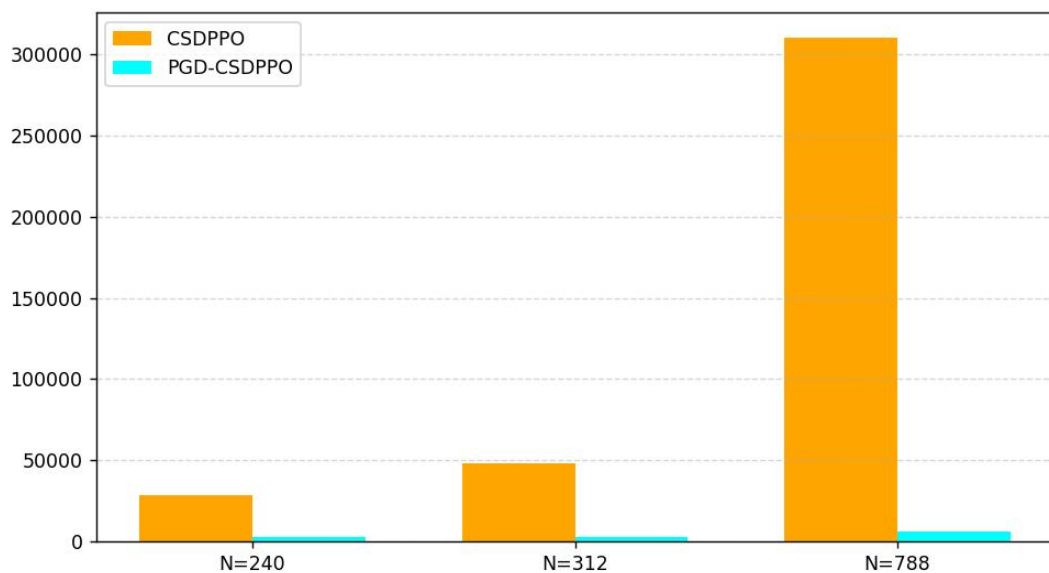
Phân tích xu hướng tăng trưởng thời gian dựa trên kết quả thực nghiệm:

Độ nhạy đối với quy mô dữ liệu: Dựa trên kết quả thu được từ các tập dữ liệu mô phỏng không gian hai chiều (Flame, Spiral và Aggregation), khi quy mô dữ liệu N tăng dần, thời gian xử lý của thuật toán CSDPPO gốc xuất hiện xu hướng tăng trưởng rất nhanh theo hàm bậc hai $O(N^2)$. Ngược lại, phương pháp cải tiến PGD-CSDPPO thể hiện sự ổn định khi thời gian có xu hướng tăng trưởng tuyến tính.

Ảnh hưởng của số chiều không gian (n): Khi đối chiếu hiệu năng giữa tập dữ liệu Iris ($n = 4$), kết quả thực nghiệm cho thấy tốc độ cải thiện thời gian của PGD-CSDPPO có phần chậm lại trên bộ dữ liệu Iris. Điều này hoàn toàn phù hợp với các phân tích lý thuyết về mặt toán học ở mục 3.2.5.

4.4.2. So sánh số lượng phép so sánh khoảng cách

Để làm rõ nguyên nhân cốt lõi dẫn đến sự cải thiện về mặt thời gian xử lý của phương pháp PGD-CSDPPO tại mục 4.4.1, thực nghiệm tiến hành thống kê tổng số lượng phép so sánh khoảng cách thực tế mà hệ thống phải thực hiện. Kết quả so sánh số phép toán giữa thuật toán gốc và phương pháp cải tiến, cùng tỷ lệ cắt tỉa không gian thành công được chi tiết hóa trong Bảng 4.4. Biểu đồ cột nhóm so sánh khối lượng phép tính được thể hiện qua Hình 4.5.



Hình 4.5. Đồ thị so sánh số phép so sánh khoảng cách

Hình 4.5 phản ánh khối lượng tính toán khổng lồ đã được tinh giản.

Bảng 4.4: Thống kê số lượng phép so sánh khoảng cách và tỷ lệ cắt tỉa

Datasets	Thuật toán	Số phép so sánh	1 – (Số phép so sánh vs Original) %	Số lượng mẫu (N)	Số chiều	No. Clusters
Flame	Gốc	28680	91%	240	2	2
	Cải tiến	2651				2
Spiral	Gốc	48516	94%	312	2	3
	Cải tiến	3134				3
Aggregation	Gốc	310078		788	2	7

	Cải tiến	6253	98%			7
Iris	Gốc	11175		150	4	3
	Cải tiến	4100	63%			3

❖ Phân tích và đánh giá hiệu quả cắt tía của phương pháp đề xuất:

Năng lực tối ưu hóa số phép toán: Ở thuật toán CSDPPO truyền thống, số phép so sánh điều kiện mật độ và khoảng cách tăng trưởng theo cấp số cộng. Đối với các điểm dữ liệu nằm ở vùng mật độ thấp, số lượng ứng viên phải quét tuyến tính ngược là cực kỳ lớn. Trong khi đó, phương pháp PGD-CSDPPO đã kiểm soát tốt sự bùng nổ chi phí này. Nhờ cơ chế duyệt có định hướng theo bán kính tăng dần, phương pháp này loại bỏ hàng loạt các ô lưới ở các tầng bán kính xa hơn mà không cần duyệt chi tiết các điểm bên trong.

Mối tương quan giữa tỉ lệ cắt tía và phân bố hình học: Kết quả thực nghiệm cho thấy tỉ lệ cắt tía đạt giá trị tối ưu nhất trên các tập dữ liệu có cấu trúc phân cụm rõ ràng như Spiral và Aggregation. Tại các bộ dữ liệu này, mật độ phân bố tập trung theo các đường biên hình học chuyên biệt. Khi cấu trúc lưới phân hoạch không gian thành các phân vùng nhỏ, các ô rỗng hoặc ô chứa các cụm hoàn toàn biệt lập ở sẽ bị chặn đứng. Hệ thống chỉ phải tốn chi phí tính toán khoảng cách thực tế đối với các phần tử nằm chung ô lưới gốc hoặc các ô lân cận trực tiếp. Tỉ lệ cắt tía cao này là minh chứng thực nghiệm rõ ràng nhất cho thấy phương pháp PGD đã giải quyết thành công bài toán vấn đề hiệu năng cho bước xác định khoảng cách δ của mô hình cũ.

4.4.3. So sánh kết quả phân cụm

Bên cạnh mục tiêu tối ưu hóa về mặt hiệu năng và giảm chi phí tính toán, một yêu cầu bắt buộc và mang tính quyết định đối với phương pháp PGD-CSDPPO là phải bảo toàn chính xác kết quả phân cụm.

Để đảm bảo an toàn và chứng minh cơ sở toán học của phương pháp cải tiến PGD-CSDPPO không làm mất mát nghiệm tối ưu, thực nghiệm tiến hành tính toán hai chỉ số đánh giá chất lượng phân cụm độc lập là ARI và NMI dựa trên nhãn thực tế (Ground Truth) của các bộ dữ liệu. Các giá trị đo lường được tổng hợp chi tiết trong Bảng 4.5 sau:

Bảng 4.5: Đối chiếu chất lượng phân cụm (ARI và NMI) giữa hai thuật toán

Datasets	Thuật toán	ARI	NMI	Sai số tuyệt đối	Số lượng mẫu (N)	Số chiều	No. Clusters
Flame	Gốc	1.0000	1.0000	0.0	240	2	2

	Cải tiến	1.0000	1.0000	0.0			2
Spiral	Gốc	1.0000	1.0000	0.0	312	2	3
	Cải tiến	1.0000	1.0000	0.0			3
Aggregation	Gốc	0.9978	0.9957	0.0	788	2	7
	Cải tiến	0.9978	0.9957	0.0			7
Iris	Gốc	0.8343	0.8244	0.0	150	4	3
	Cải tiến	0.8343	0.8244	0.0			3

❖ Phân tích và đánh giá mức độ bảo toàn cấu trúc dữ liệu:

Tính chính xác và sự trùng khớp của các chỉ số: Kết quả thực nghiệm cho thấy tại tất cả các tập dữ liệu thử nghiệm, hai chỉ số ARI và NMI của hai phương pháp CSDPPO và PGD-CSDPPO luôn trùng khớp hoàn toàn với sai số tuyệt đối bằng 0.0. Điều này chứng minh rằng, dù hệ thống đã thực thi cắt bỏ hàng loạt vùng không gian lưới và dừng sớm vòng lặp, thuật toán cải tiến vẫn không bỏ sót bất kỳ một điểm ứng viên mật độ tiềm năng nào. Cặp tham số mật độ cục bộ ρ và khoảng cách δ của từng điểm dữ liệu được bảo toàn nguyên vẹn, giữ đúng cấu trúc của biểu đồ quyết định (Decision Graph)

Ý nghĩa khoa học của chiến lược cắt tĩa hình học: Sự thống nhất về mặt kết quả phân cụm khẳng định tính đúng đắn và an toàn của cơ chế cắt tĩa dựa trên khoảng cách tối thiểu đến hộp bao Bounding Box. Bộ lọc hình học này chỉ hoạt động khi một ô lưới chắc chắn không thể chứa nghiệm tốt hơn hiện tại. Do đó, phương pháp PGD không phải là một giải thuật xấp xỉ (Heuristic) chấp nhận đánh đổi độ chính xác để lấy tốc độ, mà là một giải thuật tối ưu hóa chính xác.

4.4.4. So sánh PGD-CSDPPO với giải thuật SDPC (2024)

Trong quá trình nghiên cứu các hướng cải tiến cho thuật toán phân cụm đỉnh mật độ, đề tài cũng tham khảo nghiên cứu của Jian Hou và các cộng sự công bố năm 2024 về thuật toán SDPC (Sequential Density Peak Clustering) [19]. Đây là một hướng tiếp cận mới trong bài toán phân cụm mật độ, tuy nhiên mục tiêu cải tiến và phương pháp xử lý của SDPC khác biệt đáng kể so với phương pháp PGD được đề xuất trong đề tài.

Điểm tương đồng giữa hai phương pháp là đều lựa chọn điểm có mật độ lớn trong tập dữ liệu để làm trung tâm mở rộng cụm. Từ các điểm lõi này, thuật toán

tiếp tục thực hiện quá trình lan truyền nhằm hình thành các cụm dữ liệu hoàn chỉnh.

Tuy nhiên, mục tiêu chính của thuật toán SDPC là tự động xác định số lượng cụm K . Trong quá trình mở rộng cụm, SDPC xây dựng tập các điểm lân cận trực tiếp nằm ở biên ngoài của cụm hiện tại, tạo thành một lớp vỏ mỏng bao quanh vùng mật độ cao. Thuật toán thực hiện duyệt tuần tự các điểm thuộc lớp biên này và dừng quá trình mở rộng khi phát hiện ranh giới mật độ giữa các cụm nhằm tránh hiện tượng lan sang cụm lân cận.

Cách tiếp cận của SDPC tập trung chủ yếu vào chiến lược mở rộng cụm và phát hiện biên mật độ. Tuy nhiên, thuật toán chưa đưa ra cơ chế tối ưu hóa số lượng phép tính khoảng cách hoặc giảm chi phí truy xuất dữ liệu không gian trong quá trình xử lý.

Ngược lại, phương pháp PGD-CSDPPO trong đề tài tập trung vào bài toán tối ưu hiệu năng tính toán, đặc biệt ở bước xử lý khoảng cách hình học giữa các điểm dữ liệu. Thay vì thực hiện tìm kiếm tuần tự trên từng điểm lân cận trực tiếp, phương pháp đề xuất tiến hành phân hoạch không gian dữ liệu thành cấu trúc lưới (Grid Partitioning).

Trong quá trình mở rộng bán kính tìm kiếm r , thuật toán áp dụng cơ chế cắt tia không gian dựa trên Bounding Box để ước lượng khoảng cách tối thiểu giữa điểm truy vấn và các ô lưới lân cận. Những ô lưới nằm ngoài phạm vi tiềm năng sẽ bị loại bỏ ngay từ đầu mà không cần truy xuất các điểm dữ liệu bên trong. Nhờ đó, số lượng phép tính khoảng cách được giảm đáng kể, góp phần cải thiện thời gian thực thi trên các tập dữ liệu có kích thước lớn.

Từ các phân tích trên có thể thấy rằng:

- SDPC tập trung vào cải thiện chiến lược mở rộng cụm và tự động xác định số lượng cụm;
- Trong khi đó, PGD-CSDPPO tập trung vào tối ưu hóa hiệu năng xử lý thông qua cơ chế phân hoạch không gian và cắt tia dữ liệu.

Do đó, mặc dù đều thuộc nhóm phương pháp phân cụm đỉnh mật độ, hai hướng tiếp cận này giải quyết các khía cạnh khác nhau của bài toán phân cụm dữ liệu.

Do sự khác biệt về mục tiêu thiết kế và cơ chế hoạt động, đề tài hiện chưa thực hiện thực nghiệm so sánh trực tiếp giữa hai thuật toán trong cùng điều kiện môi trường và bộ dữ liệu. Vì vậy, phần trình bày này chủ yếu mang tính chất phân tích và đối chiếu về mặt phương pháp tiếp cận thay vì đánh giá định lượng về hiệu năng.

4.5. Thảo luận kết quả

Kết quả thực nghiệm cho thấy hiệu quả vận hành của phương pháp PGD có mối liên hệ mật thiết với đặc trưng phân bố không gian và quy mô của từng tập dữ liệu cụ thể.

Đối với các tập dữ liệu mô phỏng không gian thấp, khi quy mô mẫu N đạt ngưỡng trung bình lớn, phương pháp PGD-CSDPPO thể hiện ưu thế về năng lực giảm thiểu số phép so sánh khoảng cách. Xu hướng này có thể được lý giải là do dữ liệu phân bố tập trung theo các hình dáng cụm chuyên biệt.

Đối với các tập dữ liệu quy mô nhỏ nhưng số chiều lớn như Iris, thực nghiệm ghi nhận thời gian xử lý của PGD-CSDPPO có phần gần bằng và đôi lúc cao hơn so với thuật toán gốc, mặc dù phép so sánh khoảng cách vẫn giảm. Hiện tượng này là hoàn toàn phù hợp với lý thuyết cấu trúc dữ liệu, khi cỡ mẫu quá nhỏ, chi phí tính toán của giải thuật gốc là không đáng kể. Trong khi đó, phương pháp PGD phải tốn chi phí cố định cho việc phân hoạch ô lưới, quản lý danh sách mảng động và tính toán khoảng cách biên. Khi lợi ích từ việc cắt tỉa không đủ bù đắp cho chi phí quản lý cấu trúc hỗ trợ, tốc độ thực tế của giải thuật cải tiến có xu hướng bị chậm lại.

Một khía cạnh quan trọng khác được xác thực thông qua thực nghiệm là tính bảo toàn cấu trúc dữ liệu của phương pháp đề xuất. Việc các chỉ số đánh giá chất lượng phân cụm ARI và NMI của phương pháp PGD-CSDPPO trùng khớp hoàn toàn với thuật toán gốc CSDPPO gốc trên mọi bộ dữ liệu thử nghiệm đã minh chứng cho độ tin cậy. Kết quả này khẳng định PGD là giải thuật tối ưu hóa chính xác, không làm thay đổi các tham số, kết quả phân cụm thực tế.

Mặc dù đạt được kết quả khả quan trên các tập dữ liệu không gian thấp, hiệu năng của phương pháp PGD bộc lộ những hạn chế nhất định khi số chiều dữ liệu gia tăng. Khi thử nghiệm trên không gian nhiều chiều, xu hướng chênh lệch về mặt thời gian giữa cải tiến và giải thuật gốc có dấu hiệu thu hẹp. Hiện tượng này xảy ra do sự bùng nổ tổ hợp số lượng ô lưới lân cận theo hàm mũ ($3^n - 1$) và sự gồng chông của các hình hộp bao đa chiều, khiến xác suất kích hoạt của bộ lọc cắt tỉa giảm xuống. Điều này chỉ ra rằng phương pháp này chưa đạt trạng thái tối ưu đối với các tập dữ liệu thế giới thực có số chiều lớn và phân bố phức tạp.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Sau một thời gian tập trung nghiên cứu lý thuyết, đề xuất giải pháp cải tiến và tiến hành thực nghiệm đối chiếu so sánh, đề tài đồ án tốt nghiệp “Nghiên cứu, cài đặt và cải tiến thuật toán phân cụm đỉnh mật độ dựa trên tối ưu tham số” đã hoàn thành tất cả mục tiêu, nhiệm vụ nghiên cứu đặt ra ban đầu. Các kết quả đạt được mang ý nghĩa cả về mặt khoa học lẫn thực tiễn.

Về mặt xây dựng nền tảng lý thuyết. Đồ án đã hệ thống hóa một cách mạch lạc, tường minh các phương pháp phân cụm dữ liệu hiện nay, làm rõ ưu nhược điểm của từng hướng tiếp cận. Đặc biệt, nghiên cứu đã đi sâu bóc tách cấu trúc toán học của thuật toán phân cụm đỉnh mật độ gốc CFSFDP và phiên bản tối ưu hóa tham số khoảng cách cắt bằng Entropy (CSDPPO), từ đó chỉ ra được vấn đề về mặt hiệu năng hệ thống nằm ở chi phí tính toán $O(N^2)$ tại bước xác định khoảng cách δ .

Đồ án đã xây dựng thành công mô hình lý thuyết cho phương pháp Pruned Grid Delta (PGD) dựa trên việc kết hợp cấu trúc dữ liệu lưới phân hoạch không gian và chiến lược cắt tỉa hình học bằng Bounding Box.

Về mặt thực nghiệm và ứng dụng, nghiên cứu đã hiện thực hóa giải thuật cải tiến bằng ngôn ngữ lập trình Python và tiến hành thử nghiệm trên hai nhóm dữ liệu: tập dữ liệu mô phỏng không gian và tập dữ liệu thế giới thực. Kết quả thực nghiệm cho thấy phương pháp PGD có khả năng giảm thiểu đáng kể số lượng phép so sánh khoảng cách thực tế. Trên các tập dữ liệu có quy mô mẫu trung bình lớn và số chiều không gian thấp, giải thuật cải tiến ghi nhận xu hướng tối ưu hóa thời gian xử lý thực tế so với thuật toán gốc CSDPPO truyền thống.

Thực nghiệm đồng thời xác nhận tính an toàn về mặt toán học của cơ chế cắt tỉa. Các chỉ số chất lượng phân cụm ARI và NMI của phương pháp PGD-CSDPPO trùng khớp hoàn toàn với thuật toán gốc, chứng minh giải thuật đã bảo toàn cấu trúc thực tế không làm mất mát nghiệm tối ưu.

5.1.1. Các đóng góp chính của đề tài

Đề tài này đã đạt được một số đóng góp chính về mặt lý thuyết và thực nghiệm như sau:

- Hệ thống hóa cơ sở lý thuyết liên quan đến bài toán phân cụm dữ liệu, đặc biệt là các phương pháp phân cụm dựa trên mật độ và thuật toán phân cụm đỉnh mật độ CFSFDP.
- Phân tích nguyên lý hoạt động của thuật toán CSDPPO, đồng thời làm rõ vai trò của hàm Entropy trong quá trình tối ưu tham số khoảng cách cắt d_c .

- Phân tích và xác định được hạn chế chính về hiệu năng của thuật toán CSDPPO nằm ở bước xác định khoảng cách δ , khi số lượng phép so sánh khoảng cách tăng rất lớn theo kích thước dữ liệu.
- Đề xuất phương pháp PGD (Pruned Grid Delta) dựa trên sự kết hợp giữa cấu trúc dữ liệu lưới (grid) và cơ chế cắt tia không gian bằng Bounding Box nhằm giảm số lượng phép so sánh khoảng cách không cần thiết trong quá trình tìm kiếm điểm có mật độ cao hơn gần nhất.
- Hiện thực hóa thành công thuật toán cải tiến bằng ngôn ngữ lập trình Python và tiến hành thực nghiệm trên nhiều nhóm dữ liệu khác nhau.
- Kết quả thực nghiệm cho thấy phương pháp PGD có khả năng cải thiện hiệu năng xử lý, đặc biệt trên các tập dữ liệu có số chiều thấp và kích thước dữ liệu lớn, đồng thời vẫn bảo toàn chất lượng phân cụm so với thuật toán CSDPPO ban đầu thông qua các chỉ số đánh giá ARI và NMI.

5.1.2. Hạn chế của đề tài

Mặc dù đã đạt được một số kết quả tích cực trong việc cải thiện hiệu năng xử lý của thuật toán phân cụm đỉnh mật độ, đề tài vẫn còn tồn tại một số hạn chế nhất định cần tiếp tục nghiên cứu và hoàn thiện trong tương lai.

- Phương pháp PGD hiện hoạt động hiệu quả hơn trên các tập dữ liệu có số chiều thấp hoặc trung bình. Khi số chiều dữ liệu tăng cao, số lượng ô lưới sinh ra trong không gian dữ liệu cũng tăng nhanh, làm giảm hiệu quả của cơ chế pruning.
- Kích thước ô lưới trong mô hình hiện tại vẫn phụ thuộc vào hệ số tham số k được thiết lập thủ công. Việc lựa chọn giá trị chưa phù hợp có thể ảnh hưởng đến hiệu quả cắt tia và hiệu năng tổng thể của thuật toán.
- Phạm vi thực nghiệm của đề tài chủ yếu tập trung trên các tập dữ liệu có quy mô vừa và một số bộ dữ liệu thực tế phổ biến. Đề tài chưa tiến hành đánh giá trên các hệ thống dữ liệu cực lớn hoặc môi trường dữ liệu phân tán.
- Nghiên cứu hiện tại chủ yếu tập trung vào việc tối ưu số lượng phép so sánh khoảng cách và thời gian xử lý, trong khi các yếu tố khác như chi phí bộ nhớ, khả năng mở rộng và hiệu năng trên môi trường tính toán song song vẫn chưa được phân tích đầy đủ.
- Phương pháp đề xuất hiện mới được triển khai và kiểm thử trên CPU với môi trường thực nghiệm đơn máy, chưa khai thác các kỹ thuật tăng tốc phần cứng như GPU hoặc xử lý song song đa luồng.

5.2. HƯỚNG PHÁT TRIỂN

Từ những kết quả đạt được cùng với các giới hạn vận hành của giải thuật đã được nhận diện trong quá trình thực nghiệm. Đề tài định hướng một số nội dung nghiên cứu tiếp theo nhằm hoàn thiện và tối ưu hóa phương pháp PGD như sau:

- Nghiên cứu cơ chế kích thước ô lưới động: Trong phạm vi hiện tại, giải thuật đang áp dụng một hệ số k đồng nhất cho tất cả các chiều không gian. Trong giai đoạn tiếp theo, nghiên cứu hướng tới việc phát triển cơ chế tự động điều chỉnh kích thước ô lưới dựa trên mật độ phân bố của dữ liệu, nhằm hạn chế hiện tượng sinh ra quá nhiều ô lưới rỗng khi dữ liệu phân bố không đồng đều.
- Thử nghiệm trên các tập dữ liệu thực tế quy mô lớn hơn: Định hướng tiếp theo của đề tài là triển khai thực nghiệm giải thuật trên các tập dữ liệu thế giới thực có quy mô lớn hơn. Việc thử nghiệm này sẽ giúp đánh giá toàn diện hơn năng lực của phương pháp PGD trong việc giải quyết vấn đề chi phí tính toán của bài toán phân cụm đỉnh mật độ trong môi trường sản xuất thực tế.
- Nghiên cứu tối ưu cho dữ liệu nhiều chiều: Khi số chiều dữ liệu tăng cao, hiệu quả của cấu trúc grid có xu hướng suy giảm do hiện tượng phân mảnh không gian dữ liệu. Vì vậy, một hướng nghiên cứu tiềm năng là kết hợp thêm các kỹ thuật giảm chiều dữ liệu hoặc xây dựng cơ chế grid thích nghi cho dữ liệu nhiều chiều.
- Kết hợp xử lý song song và tăng tốc phần cứng: Một hướng phát triển khác là triển khai song song hóa quá trình tính toán khoảng cách và tìm kiếm điểm có mật độ cao hơn gần nhất. Việc kết hợp phương pháp PGD với GPU hoặc các mô hình xử lý song song có thể giúp cải thiện đáng kể hiệu năng trên các tập dữ liệu lớn.
- Mở rộng ứng dụng của phương pháp PGD: Ngoài bài toán phân cụm đỉnh mật độ, cơ chế pruning bằng grid và Bounding Box có thể được nghiên cứu áp dụng cho các bài toán xử lý dữ liệu khác như tìm kiếm lân cận gần nhất, phát hiện bất thường hoặc tối ưu truy vấn không gian dữ liệu.

TÀI LIỆU THAM KHẢO

Tiếng Việt:

[3] THÂN QUANG KHOÁT, NGUYỄN THỊ KIM ANH, ĐỖ TIẾN DŨNG, NGÔ VĂN LINH, and NGUYỄN ĐỨC ANH, “NHẬP MÔN HỌC MÁY VÀ KHAI PHÁ DỮ LIỆU”.

[4] Vũ Hữu Tiếp, “Machine Learning cơ bản”.

[5] “Phân cụm dữ liệu – Bách khoa Toàn thư Việt Nam.” Accessed: Jun. 09, 2026. [Online]. Available:

https://bkktt.vn/Ph%C3%A2n_c%E1%BB%A5m_d%E1%BB%AF_li%E1%BB%87u

Tiếng Anh:

[1] I. H. Witten, E. Frank, and M. A. Hall, *Data mining: practical machine learning tools and techniques*, 3rd ed. in Morgan Kaufmann series in data management systems. Burlington, MA: Morgan Kaufmann, 2011.

[2] P.-N. Tan, M. Steinbach, and V. Kumar, *Introduction to data mining*, New internat. edition. in Always learning. Harlow: Pearson, 2014.

[6] J. Han, M. Kamber, and J. Pei, *Data mining: concepts and techniques*, 3rd ed. in Morgan Kaufmann series in data management systems. Amsterdam Boston: Elsevier/Morgan Kaufmann, 2012.

[7] M. Ester, H.-P. Kriegel, and X. Xu, “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”.

[8] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander, “OPTICS: Ordering Points To Identify the Clustering Structure”.

[9] A. Rodriguez and A. Laio, “Clustering by fast search and find of density peaks,” *Science*, vol. 344, no. 6191, pp. 1492–1496, Jun. 2014, doi: 10.1126/science.1242072.

[10] R. P. Adams, “K-Means Clustering and Related Algorithms”.

[11] Lưu Quang Linh, “Clustering là gì? Phân loại, thuật toán và ứng dụng.” Accessed: Jun. 16, 2026. [Online]. Available: <https://devwork.vn/blog/clustering-la-gi>

[12] Harshita Dwivedi, “Phân cụm K-means và trường hợp sử dụng của nó trong Miền bảo mật.” Accessed: Jun. 16, 2026. [Online]. Available: <https://vn.linkedin.com/pulse/k-means-clustering-its-use-case-security-domain-harshita-kumari?tl=vi>

- [13] J. Wilkins, "Image Segmentation With K-Means Clustering," Towards Data Science. Accessed: Jun. 16, 2026. [Online]. Available: <https://towardsdatascience.com/image-segmentation-with-k-means-clustering-1bc53601f033/>
- [14] Z. Lv, J. Wang, X. Shi, Y. Zhuang, and S. Ge, "Clustering by Fast Searching Density Peaks Based on Parameter Optimization," in *Proceedings of the 2017 5th International Conference on Mechatronics, Materials, Chemistry and Computer Engineering (ICMMCCE 2017)*, Chongqing, China: Atlantis Press, 2017. doi: 10.2991/icmmcce-17.2017.270.
- [15] R. X. F. Chen, "A Brief Introduction to Shannon's Information Theory," Apr. 23, 2021, *arXiv*: arXiv:1612.09316. doi: 10.48550/arXiv.1612.09316.
- [16] H. Samet, *Foundations of multidimensional and metric data structures*. in The Morgan Kaufmann series in computer graphics and geometric modeling. Amsterdam ; Boston: Elsevier/Morgan Kaufmann, 2006.
- [17] W. McKinney, "Python for Data Analysis".
- [18] 米兰 M. P. / M. /, *milaan9/Clustering-Datasets*. (May 11, 2026). Accessed: May 30, 2026. [Online]. Available: <https://github.com/milaan9/Clustering-Datasets>
- [19] J. Hou, H. Lin, H. Yuan, and M. Pelillo, "Flexible density peak clustering for real-world data," *Pattern Recognit.*, vol. 156, p. 110772, Dec. 2024, doi: 10.1016/j.patcog.2024.110772.